A Guide to Swift Concurrency

**A field guide and reference** · Draft 2.0 · 2026-07-08 · Jonathan Gilbert

**Covers:** Swift 5.5 through Swift 6.4 (beta), every accepted Swift Evolution concurrency proposal, and the rules this project should adopt to keep the whole system sound as it grows.

> **A note on standing.** The Swift project publishes no normative requirements document for its concurrency model. This guide reconstructs one from the proposals, the vision documents, and the compiler itself. Where a requirement is stated by the project, it is cited in Appendix D. Where it is an inference—something the design plainly relies on but never wrote down—it is marked **[unwritten]**, because the unwritten ones turn out to matter most. The history of how this document came to exist, and the analysis of where the current implementation falls short of its own intentions, live in the appendices so that the main text can simply teach you the system as it is meant to work.

---

## Table of contents



---

## Part 1: The big picture

The deepest idea in computing is not the algorithm, and it is not the data structure. It is the *boundary*—the membrane that separates one context from another, and the discipline governing what may pass through. A biological cell works because its membrane is picky. A great software system works for the same reason. When people say "object-oriented," what they should mean—what the term was coined to mean—is not classes and inheritance but *messaging*: little self-contained contexts that never reach into each other's insides, communicating only by sending things across well-defined boundaries.

Swift concurrency is that idea, taken seriously, and enforced by a compiler.

Here is the picture to hold in your mind for the rest of this guide. A running program is a **building**. Inside it are **rooms**—the isolation domains. Each actor instance is a room. Each global actor is a room; the biggest and most famous is the **front desk**, the main actor, which is where the user-facing work happens. Mutable state lives in rooms, and each piece of state lives in exactly one.

Every room has a single door and a **receptionist**—a serial executor—who admits one visitor at a time. You never barge into a room; you leave a message at the door and wait to be let in. That waiting is what `await` marks.

Work is carried out by **couriers**—tasks. A courier follows one errand from start to finish, stepping into rooms as needed, waiting politely at doors. Couriers hire sub-couriers for parallel legs of the errand, and a courier always waits for its sub-couriers to report back before going home. That's structured concurrency: the errands form a family tree, not a swarm.

Then there is the question of *things*. Some things are freely copyable—a memo you can photocopy and slip under any door. Those are `Sendable` values. Other things are big, stateful machines with cables running to other machines. You cannot photocopy those. You can only *move* them, and if you move one you must move everything cabled to it—the whole **box**—and once you've handed the box into a room, you keep no key to it. That is regions and `sending`.

Finally, the building has a small **hallway crew**—the cooperative thread pool—who do the roomless work. The crew is small on purpose, about one member per CPU core, and it has one iron rule: never stand still. A crew member who stops to wait for another crew member can freeze the whole building.

That's the entire model. Every keyword, attribute, and macro in Swift concurrency is a piece of this picture:

| In the analogy | In the language |
|---|---|
| A room | An isolation domain (actor instance, global actor) |
| The front desk | `@MainActor` |
| The receptionist | A serial executor |
| Waiting at the door | `await` (a suspension point) |
| The hallway (no room at all) | `nonisolated` code, the global concurrent executor |
| A courier | A task |
| Sub-couriers | Child tasks (`async let`, task groups) |
| A photocopiable memo | A `Sendable` value |
| A box of cabled-together machines | An isolation region (non-`Sendable` values that can reach each other) |
| Signing the box over, keeping no key | `sending` |
| A box whose shipping label states its home room | `@isolated(any)` |
| The hallway crew | The cooperative thread pool |

And the entire system exists to guarantee one sentence:

> **If your program compiles in the Swift 6 language mode, and you used none of the honor-system opt-outs, it has no data races.**

Everything else—some three dozen requirements, forty-odd language features, five compiler subsystems—is in service of making that sentence true, and of making it *pleasant to live with*, which turns out to be almost as hard.

One more thing before the details. A guarantee like this is not a percentage. It is not "mostly no data races." It is a chain, and a chain with one broken link holds nothing. That is why this guide ends (Part 6) with rules about how *changes* to the system must be written up: most of the real-world breaks in the chain have come not from bad code but from good code whose safety rested on a premise somebody else later changed, in a document nobody could find. A concurrency model is a promise, and promises need bookkeeping.

## Part 2: The requirements

Here is what Swift concurrency actually requires—the load-bearing rules. Each has a memorable name and a reference number, so that later parts of this guide (and, one hopes, future proposals and commit messages) can say things like "this upholds the *Move the Whole Box* requirement (2.14)" instead of pointing at folklore.

They come in seven groups: the ground rules of memory, the promise, the rules of rooms, the rules of moving things, the rules of the courier family, the rules of the crew, the rules of the border with old code, and the character requirements—the ones about what the system must *feel* like.

### The ground rules of memory

These two predate `async/await` entirely. They are the physics on which everything else is built.

**One Writer at a Time (2.1).** Two accesses to the same variable conflict unless both are reads, and conflicting accesses must never overlap in time. The compiler enforces this statically where it can and dynamically where it can't. This is the Law of Exclusivity, and it is why Swift can even *define* what a race is: without it, "simultaneous access" would be a matter of opinion. *(SE-0176)*

**Everything Needs an Order (2.2).** Swift uses the C/C++ memory model: a data race is a pair of conflicting, non-atomic accesses that are not ordered by the *happens-before* relation, and a program containing one has undefined behavior—not "a wrong answer," but *anything at all*. So "no data races" has a precise meaning: every pair of conflicting accesses, in every possible execution, is ordered. All of Swift concurrency is a machine for manufacturing happens-before edges. *(SE-0282, SE-0410)*

### The promise

**If It Compiles, It Doesn't Race (2.3).** A program that type-checks under complete concurrency checking (the Swift 6 language mode) and uses none of the honor-system opt-outs (2.37) contains no data races in the sense of *Everything Needs an Order* (2.2). This is the cornerstone. The project's own words: eliminate data races the same way Swift eliminated memory unsafety.

Every other requirement in this Part exists either to make this one true or to make it bearable.

**Prove It Before You Run It (2.4).** The guarantee is delivered at compile time by default. Runtime enforcement—deterministic traps—is reserved for the borders with unchecked code and for explicit programmer assertions. A trap is the fallback, never the plan. **[unwritten]**

**No Deadlocks by Design—Mostly (2.5).** The language's own primitives never deadlock: an actor is *re-entrant* at suspension points precisely so that awaiting into a busy actor cannot wedge. The price of that re-entrancy is the *Room May Change While You're Away* requirement (2.9). Note the promise is deliberately modest: the language will not deadlock you, but you can still deadlock yourself—by blocking a crew thread (2.23) or by taking two locks in different orders. The original 2020 roadmap promised more ("eliminate data races *and deadlocks*"); the deadlock half was quietly narrowed and never formally retracted, a small cautionary tale about unmaintained promises that Appendix A returns to. *(SE-0306)*

### The rules of rooms

**Everything Lives Somewhere (2.6).** Every declaration—every function, property, initializer, subscript—has exactly one static isolation: a specific actor instance, a global actor, the caller's own isolation, or `nonisolated` (the hallway; no room at all). There is no such thing as a "sometimes isolated" declaration. When you need code that works *for any room*, you say so explicitly, with an `isolated` parameter or `@isolated(any)`—polymorphism over rooms is a feature you ask for, never an ambiguity you fall into. *(SE-0306, SE-0313, SE-0316, SE-0420, SE-0431, SE-0461, SE-0466)*

**Only Touch What's in Your Room (2.7).** Code may synchronously access state belonging to room *D* only if the code itself is isolated to *D*. From anywhere else, you go through the door: an asynchronous call, with an `await`. There is exactly one exception—reading immutable `Sendable` (`let`) storage, which is safe from anywhere because nobody can change it. *(SE-0306)*

**One Visitor at a Time (2.8).** Every room with a serial executor guarantees that all work isolated to it is mutually excluded and totally ordered. The receptionist never double-books. If you write a *custom* executor, this guarantee becomes your personal promise—the compiler cannot check it (opt-out 2.37-f). *(SE-0306, SE-0392)*

**The Room May Change While You're Away (2.9).** Between two suspension points, your code runs without interleaving inside its room. But every `await` is a door you step out of, and while you're out, *other visitors may enter*. When you come back, the furniture may have moved. Consequence: an actor's invariants must be re-established before every `await` and re-checked after. An `await` is a transaction boundary, not a pause button. This is the single most misunderstood rule in Swift concurrency, and half the features in Part 4 exist in some relation to it. *(SE-0296, SE-0306)*

**Run Where the Label Says (2.10).** The compiler inserts executor hops so that code always *physically runs* on the executor of its static isolation—on entry to an isolated function, and again on every resumption from a suspension. Static isolation and dynamic executor must never disagree in safe code. When they do, it is not a diagnostic gap; it is a miscompile, and the runtime assertions exist to catch exactly this. **[unwritten—but it is the invariant the runtime's isolation checks probe, and its violation in the Swift 6.3.2 miscompile was treated as a critical bug]**

**No Homeless Code (2.11).** Code that runs implicitly on behalf of a declaration—default-argument expressions, stored-property initializers, `defer` bodies, deinitializers—always has a defined isolation. Nothing in a Swift program executes in an unspecified room. *(SE-0411, SE-0493, SE-0371, SE-0327)*

**Even Conformances Live Somewhere (2.12).** A protocol conformance is itself a fact that can carry isolation. A conformance isolated to the front desk may only be *used* where the front desk's protection is guaranteed, and generic code that could smuggle the conformance elsewhere must be fenced off. *(SE-0470)*

### The rules of moving things

This group is the heart of the system—and, historically, the site of every serious accident. In the analogy: what may pass through a door, and on what terms.

**Copies Travel Free (2.13).** A value may cross a room boundary if its type is `Sendable`—if handing over a copy can never create shared mutable state. `Sendable` is checked structurally: all stored properties must themselves be Sendable; classes must additionally be final with immutable state (or take the honor-system oath, 2.37-a). A `@Sendable` closure may capture only Sendable values, and only by value. *(SE-0302)*

**Move the Whole Box (2.14).** A non-`Sendable` value may still cross a boundary—*if* its entire region is disconnected from the sender's room at the moment of crossing. A region is the set of values tied to a value by aliasing or reachability: if `x` can reach `y`, they are cabled together and travel as one box. The crossing transfers the whole box, and afterward *any* use of *anything* in the box from the sending side is a compile-time error. And a box merged into a room's permanent contents stays there forever—rooms do not give things back. *(SE-0414)*

**Hand It Over and Let Go (2.15).** A `sending` parameter is a two-sided contract. The caller's obligation: the argument arrives in a disconnected box (nothing on the caller's side still holds a cable to it). The callee's right: it may merge the box into *any* room it likes—including shipping it onward somewhere else entirely. A `sending` result is the mirror image. The right to re-send is the part people forget, and forgetting it is what caused the motivating bug of this document (Appendix C). *(SE-0430)*

**When Unsure, Assume Tangled (2.16).** When the checker cannot *prove* two values are in separate boxes, it must assume they are cabled together and merge their regions. Uncertainty is always resolved toward safety: false alarms are permitted, silent unsoundness never is. A false positive is a bug against comfort (2.33); a false negative is a bug against the promise (2.3), and those two failures are not the same species. *(SE-0414)*

**Labels Never Lie (2.17).** The isolation the type system attributes to a value—above all, to a *function* value—must remain a truthful description of what that value does at runtime, through every conversion, capture, and erasure the language permits. A closure that will touch front-desk state must never end up typed as if no room were involved. **[unwritten—never stated anywhere in the project's documents, yet both the type checker's conversion rules and the region analysis silently assume it. Nearly every known soundness hole is a violation of this one requirement. See Appendix C.]**

### The rules of the courier family

**Children Finish First (2.18).** A child task cannot outlive the scope that created it. When a scope exits, it awaits all its children—or cancels them and then awaits them. An `async let` child not awaited on some path is implicitly cancelled and awaited. The errand tree never sheds leaves silently. *(SE-0304, SE-0317)*

**Cancellation Is a Request, Not a Command (2.19).** Cancellation is a one-way flag that propagates down the task tree. It never preempts, never injects an error, never yanks a courier off the street. Tasks *observe* it voluntarily—`Task.isCancelled`, `Task.checkCancellation()`, cancellation handlers—and decide how to wind down. A scope may temporarily shield itself from *observing* the flag, but the flag itself stays raised and visible outside the shield. *(SE-0304, SE-0504)*

**Urgency Flows Through (2.20).** Child tasks inherit priority, and when a high-priority courier awaits a low-priority one, the awaited task is *escalated*—urgency flows through await edges to wherever the blocking work actually is. This is what makes priorities compose without manual plumbing. *(SE-0304, SE-0462)*

**Children Inherit the Household (2.21).** Structured children and `Task { }` inherit task-local values, priority, and—for `Task { }`—the actor context they were created in. `Task.detached { }` inherits nothing; it is an orphan by design, and should be about as rare as orphans. *(SE-0311, SE-0431)*

**Resume Exactly Once (2.22).** Every continuation is resumed exactly once—not zero times (a leak: a courier waiting forever at a door that never opens), not twice (a corruption: two couriers walking out of one door). Checked continuations enforce this with traps and warnings; unsafe continuations make the same rule your problem (2.37-c). This is the boundary condition for bridging any callback-based world into the task world. *(SE-0300)*

### The rules of the crew

**Always Keep Moving (2.23).** The default runtime is a cooperative pool of roughly one thread per core, and code running on it must always make forward progress. Never block a crew thread waiting for work that needs the *same crew* to run—no semaphores signaled from other tasks, no condition variables, no blocking joins across task dependencies. The pool does not grow to cover blocked threads; that GCD-era escape hatch is deliberately absent. Violations don't make the program slow, they make it *stop*. Brief, bounded lock hold-times are fine. *(WWDC21 "Behind the scenes")*

**Stepping Aside Is Free (2.24).** Suspension parks the task's frame on the heap and frees the thread immediately. Threads are never held across an `await`. Waiting at a door costs the building nothing; only *standing in the hallway refusing to move* (2.23) costs anything.

**Order Within, Not Between (2.25).** A serial executor guarantees mutual exclusion and total order—but *not* first-in-first-out. The default actor executor is priority-aware and may serve an urgent visitor before a patient one. Never architect around an assumed mailbox order; ordering between tasks beyond happens-before is explicitly out of scope.

**You May Always Ask Where You Are (2.26).** The runtime can answer "am I in room D right now?"—`assertIsolated`, `preconditionIsolated`, and `assumeIsolated`, the last of which converts a successful runtime check into static isolation for a scope, trapping if the assumption is wrong. Custom executors can supply their own notion of "you are effectively in my room." *(SE-0392, SE-0424, SE-0471)*

### The rules of the border

Swift concurrency did not replace the old world of threads, queues, and callbacks; it annexed it. These rules govern the border crossings.

**Old Code Speaks Through Translators (2.27).** Objective-C completion-handler methods import as `async` functions; the call-the-handler-exactly-once convention maps onto return-or-throw-exactly-once. *(SE-0297)*

**Assume Strangers Call From Anywhere (2.28).** A completion-handler parameter imported from Objective-C is `@Sendable` by default, because the platform may invoke it on any thread it pleases. Unchecked code is presumed hostile until annotated otherwise. *(SE-0463)*

**Soft Borders During Migration (2.29).** Checking degrades gracefully at boundaries with unchecked modules: `@preconcurrency import` downgrades or suppresses the diagnostics that the unchecked module would otherwise trigger, and declarations vended `@preconcurrency` keep working for clients who haven't migrated. Trust is extended deliberately, and labeled. *(SE-0337)*

**Trap, Don't Race (2.30).** Wherever static checking was waived—a `@preconcurrency` conformance, for instance—the compiler inserts dynamic isolation assertions, so that a violated assumption becomes a deterministic, debuggable trap instead of a silent race. If the promise (2.3) must bend at a border, it bends toward a loud crash, never toward quiet corruption. *(SE-0423)*

**Thread-Bound Code Stays Put (2.31).** An API whose meaning depends on *thread identity*—recursive locks, thread-local storage, anything that must unlock on the same thread that locked—can be marked unavailable from async

contexts (`@available(*, noasync)`), because a task may hop threads at every suspension. *(SE-0340)*

### The character requirements

These are requirements about what the system must feel like, stated in the project's vision documents. They constrain the design as hard as the technical rules do—and the tension between them and the promise (2.3) is where nearly all the design difficulty lives.

**Simple Programs Stay Simple (2.32).** A sequential, single-threaded program must compile without ever encountering a concurrency diagnostic. Using `async`/`await` without parallelism must not force you to learn data-race safety. Only actual parallelism may surface the full machinery. Complexity must arrive with the problem, never before it.

**Annotations Are a Tax to Minimize (2.33).** The number of explicit concurrency annotations in ordinary code must be driven down relentlessly. A sound-but-noisy checker is not a success: false positives are real bugs against this requirement, even when the checker is technically right to be cautious.

**Check Locally, Never Globally (2.34).** No whole-program analysis, ever. All checking is per-declaration and per-function-body, against declared interfaces. Whatever a caller's analysis learns must be expressible in the callee's *signature*, and vice versa—the type system is the only channel between compilation units, which is precisely why *Labels Never Lie* (2.17) is load-bearing.

**Every Change Brings Its Own Movers (2.35).** Any source-breaking change to concurrency semantics ships with a migration mode that produces mechanical, semantics-preserving fixes. You may change the rules; you may not strand the residents.

**Dialects Are the Price (2.36).** The language accepts that per-module settings—language mode, upcoming-feature flags, default isolation—create real semantic dialects: the same file can mean different things under different settings. The vision documents call this cost "modest and manageable." It is manageable—*if* the semantics are written down per dialect, which is one of the jobs of the litmus-test rule in Part 6.

### The honor system (2.37)

The promise (2.3) is conditional on your not using these. Each is an oath: a proof obligation you assert and the compiler does not check.

| | Feature | The oath you swear |
|---|---|---|
| a | `@unchecked Sendable` | "This type is thread-safe by means the checker cannot see" (internal locking, etc.) |
| b | `nonisolated(unsafe)` | "This global or storage is protected by external means" |
| c | `withUnsafeContinuation` | "I will resume exactly once" (2.22, by hand) |
| d | `UnsafeCurrentTask`, unsafe pointers | Standard unsafe-Swift rules; pointers don't cross rooms |
| e | `@preconcurrency` import/decl | "The unchecked module honors the annotations I claim" (partially backstopped by *Trap, Don't Race*, 2.30) |
| f | Custom `SerialExecutor` | "My executor really admits one visitor at a time, in a total order" (2.8) |
| g | `assumeIsolated` | Safe (it traps)—but shifts a static proof to runtime |
| h | Swift 5 mode / partial strictness | Waives the promise wholesale, during migration |

Swear an oath falsely and the promise is void—not weakened, void. That is the nature of chains.

---

## Part 3: Why the requirements add up to safety

It is worth seeing, once, how the pieces compose—because it explains why every requirement above is load-bearing, and why a single unsound rule anywhere voids the whole promise.

Take any two accesses to the same memory that could conflict (at least one is a write). Ask: what *kind* of memory is it? There are only four answers, and each answer comes with its own ordering argument:

**Immutable memory.** A `let` of Sendable type, an immutable capture. No writes exist, so no conflict exists. Done.

**Room-protected memory.** Actor stored state, global-actor state, isolated globals. By *Only Touch What's in Your Room* (2.7), every access happens in code isolated to that room. By *One Visitor at a Time* (2.8), the receptionist runs those pieces of work one after another, in a total order—and "one after another" is exactly a happens-before edge. *Run Where the Label Says* (2.10) guarantees the code really is physically in the room its label claims, and *No Homeless Code* (2.11) extends the coverage to initializers, deinitializers, defaults, and defers.

**Box memory.** Everything non-Sendable that isn't actor state. *Move the Whole Box* (2.14) and *Hand It Over and Let Go* (2.15) guarantee that at any moment, each box is reachable from at most one room. Boxes move; they are never shared. And the moves themselves are ordered: task creation, task completion, and continuation resumption are all happens-before edges. So accesses before and after a hand-off are ordered, and simultaneous access is impossible because the sender lost the box.

**Synchronization memory.** `Atomic` and `Mutex` values order their own accesses; that's what they're for.

Four cases, four ordering arguments, no fifth kind of memory. Every conflicting pair is ordered; by *Everything Needs an Order* (2.2), there are no data races. That is the whole proof, and it is a *good* proof—the region half was adapted from a type system with a published, machine-checked soundness proof (Milano, Turcotti & Myers, PLDI 2022). Swift's design is not a perpetual-motion machine; on paper, it is sound.

But look at what the proof leans on. The room case assumes the code accessing room state really carries the room's label—*Labels Never Lie* (2.17). The box case assumes a box's classification as "disconnected" was computed from truthful labels—2.17 again. One unwritten requirement, load-bearing in two of the four cases. When it is violated—when a conversion quietly strips a room label off a closure—both arguments fail at once, and the promise fails with them. Appendix C shows this happening in a real, compiler-accepted, sanitizer-confirmed race, and Part 6 exists so it doesn't happen again.

Keep the four cases in mind as you read Part 4. Every feature below is a tool for putting memory *into* one of the four cases and keeping it there.

---

## Part 4: The features, one at a time

This is the reference half of the guide: every concurrency keyword, type, attribute, and macro—what it's for, when to reach for it, what misuse looks like, what the compiler says when you get it wrong, and which other features live next door to it in real code.

A word on how to read the error messages. Swift's concurrency diagnostics are verbose but almost always say one of four things, corresponding to the four cases of Part 3: *you touched a room's state from outside the room* (2.7), *you tried to copy something that isn't copyable across rooms* (2.13), *you tried to move a box you don't fully own* (2.14–2.16), or *you used something after handing it over* (2.15). Once you can classify a diagnostic into one of those four, you're halfway to the fix.

### 4.1 `async` and `await`

**What they're for.** `async` marks a function as an *errand*—a piece of work that may need to wait at doors. `await` marks each place where the waiting can actually happen: the suspension points. Between two `await`s, your code runs to completion without interleaving in its room; at each `await`, the door opens (2.9).

**Why the marking matters.** Suspension points are where the world can change underneath you. Making them syntactically visible means you can audit your invariants by eye: look at each `await`, and ask "is everything consistent right now? and do I re-check what I assumed when I resume?"

**Proper use**—treat each `await` as a transaction boundary:

```
actor BankAccount {
    var balance: Int = 0

    func transfer(amount: Int, to other: BankAccount) async throws {
        guard balance >= amount else { throw TransferError.insufficient }
        balance -= amount                    // commit our side FIRST…
        await other.deposit(amount)          // …then cross the door
    }
}
```

**Misuse**—checking an invariant on one side of an `await` and relying on it on the other:

```
func transfer(amount: Int, to other: BankAccount) async throws {
    guard balance >= amount else { throw TransferError.insufficient }
    await other.prepare()        // ⚠ door opens: another transfer may run
```

```
NOW
    balance -= amount            // 🪲 balance may no longer cover `amount`
}
```

No compiler error here—this is *logically* wrong, not race-wrong. The room changed while you were away (2.9). Re-check after every `await`, or restructure so the check and the mutation sit between the same pair of suspension points.

**Errors you'll meet and what they mean:**

```
error: expression is 'async' but is not marked with 'await'
```

You called an async function without acknowledging the door. Add `await`—and then *think about what it implies*, because the compiler is telling you an interleaving point exists here.

```
error: 'async' call in a function that does not support concurrency
```

You called an errand from a non-async function. Either make the caller `async` (propagating honesty up the call chain) or, if you're at a boundary where you genuinely must fire-and-forget, wrap it in a `Task { }`—and read the `Task` section (4.13) before you do.

**Architecture.** Make functions `async` because they *are* asynchronous—they wait for time, I/O, or other rooms—not to fashionably decorate them. A synchronous function is a stronger promise ("I complete without the world changing") and strong promises are what invariants are built from.

**Neighbors.** `await` is the mechanism behind actor calls (4.5), `async let` (4.2), and everything in the task family. Its interleaving semantics (2.9) are why `Mutex` critical sections must not contain one (4.20).

### 4.2 `async let`

**What it's for.** The lightest way to hire a sub-courier: start a child task for one value, keep doing other work, and collect the result with `await` when you need it.

**Proper use**—genuine distribution of independent work:

```
func loadProfile(id: UserID) async throws -> Profile {
    async let avatar = fetchAvatar(id: id)      // child task starts now
    async let history = fetchHistory(id: id)    // this one too, in parallel
    let name = try await fetchName(id: id)      // meanwhile, on our own task
    return try await Profile(name: name, avatar: avatar, history: history)
}
```

Both children are bounded by the scope (2.18): if `fetchName` throws, the unawaited children are automatically cancelled and awaited before the error propagates. No leaks, no orphans, nothing to remember.

**Misuse**—using `async let` as a fire-and-forget:

```
func save(_ document: Document) {
    async let _ = uploadToServer(document)   // 🚫 error: 'async let' in a function
                                             // that does not support concurrency
}
```

`async let` is *structured*: it exists to be awaited inside an async scope. For fire-and-forget you want `Task { }`—with the caveats in 4.13.

**Errors you'll meet.** Passing a non-Sendable value into an `async let` initializer can produce a region diagnostic—`sending 'document' risks causing data races`—because the child runs concurrently with you: whatever the child gets must be a memo (2.13) or a fully surrendered box (2.14). Same medicine as always: make the type Sendable, or stop touching the value after the child starts.

**Architecture.** Reach for `async let` when the number of parallel legs is *fixed and small*—two, three, five known things. When the count is dynamic, that's a task group (4.14).

**Neighbors.** `withTaskGroup` (4.14) for dynamic distribution; `Task { }` (4.13) when you truly need work outliving the scope.

### 4.3 AsyncSequence and for await

**What they're for.** Iteration where each element may take time. A `for await` loop pulls elements one at a time, suspending once per element—a door-wait per iteration.

```
for await line in url.lines {
    process(line)          // between elements, your room can change (2.9)!
}
```

**Misuse**—assuming loop-carried room state is stable across iterations. Each `await` in the loop header is a full suspension point; if you're on the main actor, UI events run between elements. Snapshot what you need, or design the loop body to tolerate change.

**Architecture.** `AsyncSequence` is the right *shape* for anything stream-like: notifications, socket messages, file lines, UI events. To manufacture one from a callback world, use `AsyncStream` (4.22). Since Swift 6, sequences carry a typed `Failure`, so `for try await` propagates errors honestly.

**Neighbors.** `AsyncStream` (4.22) makes them; cancellation (4.16) ends them early; the `Observations` API surfaces state changes as one, with anti-tearing semantics tied to suspension points (2.9).

#### 4.4 actor

**What it's for.** Declaring a room. An `actor` is a reference type whose stored state belongs to one isolation domain per instance, guarded by a serial executor. It upholds *Everything Lives Somewhere* (2.6), *Only Touch What's in Your Room* (2.7), and *One Visitor at a Time* (2.8) in a single keyword—and it is implicitly `Sendable`, because a reference to a room is just an address; the room protects itself.

**When to use it.** Whenever you have mutable state plus concurrent access. Caches, connection pools, rate limiters, download managers, game world state—anything where today you'd reach for a serial dispatch queue plus discipline, an actor gives you the queue *and* the discipline, compiler-enforced.

**Proper use:**

```
actor ImageCache {
    private var images: [URL: Image] = [:]

    func image(for url: URL) async throws -> Image {
        if let cached = images[url] { return cached }
        let image = try await downloadImage(url)   // door opens here (2.9)…
        images[url] = image                        // …so re-entry is
possible
        return image
    }
}
```

**The classic subtle bug**—actor re-entrancy. In the code above, two callers can both miss the cache, both download, and both insert. Not a data race (the promise holds; state is never *corrupted*), but duplicated work—a logic consequence of *The Room May Change While You're Away* (2.9). The pattern that fixes it is remembering the in-flight work, not the result:

```
actor ImageCache {
    private var tasks: [URL: Task<Image, Error>] = [:]
```

```
    func image(for url: URL) async throws -> Image {
        if let existing = tasks[url] { return try await existing.value }
        let task = Task { try await downloadImage(url) }
        tasks[url] = task                          // recorded BEFORE any
await
        return try await task.value
    }
}
```

**Misuse**—trying to touch a room's state from the hallway:

```
actor Counter { var value = 0 }

func bump(_ c: Counter) {
    c.value += 1
    // 🛑 error: actor-isolated property 'value' can not be
    //    mutated from a nonisolated context
}
```

The fix is never "find a way in"; it's to *ask the room to do it*—add a `func increment()` to the actor and `await c.increment()`. If you find yourself writing many tiny getters and awaiting them in sequence, that's the second classic mistake:

```
// 🍾 Race-free but WRONG: three separate visits; room changes between them
let count = await stats.count
let sum = await stats.sum
let mean = Double(sum) / Double(count)   // count and sum may be from
different eras
```

Make one visit that computes the answer inside:

```
let mean = await stats.mean()            // one visit, one consistent
snapshot
```

**Errors you'll meet:**

```
error: actor-isolated property 'x' can not be referenced from a nonisolated
context
```

You're outside the room (2.7). Go through the door: make the access `awaited` and async, or move the code into the actor.

```
error: sending 'result' risks causing data races
note: actor-isolated 'result' is returned from an actor method and cannot
cross...
```

You tried to hand a caller something still cabled into the room (2.14). Return a `Sendable` snapshot (a value-type copy), or declare the result `sending` and build it fresh inside the method so it's a disconnected box.

**Architecture.** Think of actors as *service providers*, not as objects-with-a-lock. Design their public methods as complete, meaningful transactions ("transfer", "checkout", "resolve") rather than property-level pokes; each method is one visit, and one visit is your atomicity unit. Keep rooms coarse enough that operations are transactions, fine enough that the receptionist isn't a bottleneck. And prefer *value types in, value types out*: memos through the door, machinery stays inside.

**Neighbors.** `@MainActor` (4.5) is a global room; `nonisolated` (4.6) pokes hallway-safe holes in an actor's wall; `isolated` parameters (4.7) let free functions run inside a room; custom executors (4.18) replace the receptionist.

### 4.5 `@MainActor` and `@globalActor`

**What they're for.** A global actor is one room for the whole process, addressable by type name from anywhere. `MainActor` is the one you'll use daily: its executor is the main thread, making `@MainActor` the type-checked replacement for "must be called on the main thread" comments. The correspondence is exact by construction: the main actor's executor drains the main dispatch queue.

**When to use it.** UI state, view models, and anything that must observe main-thread discipline. Annotate a whole type when it lives at the front desk:

```
@MainActor
final class ProfileViewModel {
    var profile: Profile?
    var isLoading = false

    func load(id: UserID) async {
        isLoading = true
        defer { isLoading = false }
        profile = try? await profileService.fetch(id: id)   // hop away and
back
    }
}
```

Note what's absent: no `DispatchQueue.main.async`, no "am I on the main thread?" anxiety. `load` is front-desk code; the `await` hops to the service and *the resumption hops back automatically* (2.10).

**Misuse #1**—sprinkling `@MainActor` to silence diagnostics. Every annotation is a claim about where code *belongs*. If you put `@MainActor` on a parser because a

diagnostic went away, you just scheduled your parsing on the UI's receptionist. The front desk is for front-desk work.

**Misuse #2**—the "escape to the main queue" reflex:

```
// 🪰 Old-world reflex inside the new world
func updateUI() {
    DispatchQueue.main.async { self.label.text = "Done" }  // invisible to
checker
}

// ✅ Say what you mean; let the compiler check it
@MainActor func updateUI() {
    label.text = "Done"
}
```

**Errors you'll meet:**

```
error: main actor-isolated property 'profile' can not be mutated from a
       nonisolated context
```

Same as any room violation (2.7): get into the room. From async code, `await MainActor.run { ... }` or call a `@MainActor` function. From synchronous code that you *know* is on the main thread (a legacy callback), `MainActor.assumeIsolated { ... }`—which traps if you're wrong (2.26), which is exactly what you want (2.30).

```
error: call to main actor-isolated instance method 'render()' in a
synchronous
       nonisolated context
```

A synchronous function can't wait at doors. Make the caller async, or make the caller `@MainActor` too if it belongs at the front desk.

**Custom global actors** are for when a *subsystem* needs its own process-wide room—an audio engine, a database connection with thread affinity:

```
@globalActor
actor AudioActor {
    static let shared = AudioActor()
}

@AudioActor func startEngine() { ... }   // all @AudioActor code shares one
room
```

Create one only when many types across many files must share one serialization domain. If a single type needs protection, a plain `actor` is the right size.

**Architecture.** A good app has a *small number of named rooms*: the main actor for presentation, perhaps one or two subsystem actors, instance actors for genuinely independent stateful services. If you can't say in one sentence what a room protects, it shouldn't exist.

**Neighbors.** Default isolation (4.10) can make `@MainActor` the module-wide default so single-threaded programs stay simple (2.32); `assumeIsolated` (4.19) is the border tool; isolated conformances (4.12) let front-desk types conform to nonisolated protocols honestly.

### 4.6 `nonisolated`, `nonisolated(unsafe)`, and `~Sendable`

**What `nonisolated` is for.** Declaring hallway code: a declaration that belongs to *no* room. It can run anywhere, anytime—and precisely because of that, it may synchronously touch only immutable `Sendable` state. It's how you poke a safe hole in an actor's wall for things that need no protection:

```
actor Player {
    let id: PlayerID                        // immutable: safe from anywhere
    var score = 0                           // mutable: room-protected

    nonisolated var displayName: String { // callable synchronously, no await
        "Player \(id)"                      // touches only immutable state ✅
    }
}
```

This is what lets an actor conform to `CustomStringConvertible`, `Hashable`, or `Codable` requirements that the protocol declares as synchronous.

**Misuse:**

```
nonisolated var summary: String {
    "Player \(id): \(score)"
    // 🚫 error: actor-isolated property 'score' can not be referenced
    //    from a nonisolated context
}
```

Hallway code gets no key to the room—even the room it's declared in.

**`nonisolated` on types and extensions** cuts off global-actor *inference*: inside a `nonisolated extension`, members stop inheriting `@MainActor` from conformances or enclosing context. Useful for carving hallway-safe helper zones out of front-desk types, in service of fewer annotations (2.33).

**`nonisolated(unsafe)`** is honor-system oath 2.37-b: "this mutable global/static/storage is protected by means you can't see." Every use should sit next to the thing that actually protects it, and a comment saying so:

```
// Protected by `stateLock`—all access goes through withState(_:) below.
nonisolated(unsafe) private var _state: EngineState
private let stateLock = NSLock()
```

If you cannot write that comment truthfully, you have found a bug, not an annotation site. Prefer `Mutex` (4.20), which proves the same thing to the compiler instead of asserting it.

**`~Sendable`** is the opposite hygiene tool: it *suppresses* Sendable inference on a type that would otherwise earn it structurally, because you know instances are about to grow un-shareable guts (a file handle, a cache) and you don't want clients depending on a conformance you intend to revoke. Declaring capability you don't promise is how future soundness debt starts; `~Sendable` is how you decline politely.

**Neighbors.** `nonisolated(nonsending)` (4.8) is a different beast despite the shared spelling—read it next. `Sendable` (4.11) defines what hallway code may touch.

### 4.7 `isolated` parameters and `#isolation`

**What they're for.** Room-*polymorphic* functions—code that runs inside whatever room you hand it. This is the explicit polymorphism that *Everything Lives Somewhere* (2.6) promises: never ambient, always declared.

```
func withLogging<T>(
    on actor: isolated any Actor,        // this function runs INSIDE `actor`
    _ body: () throws -> T               // so it can call sync isolated code
) rethrows -> T {
    log("entering")
    defer { log("exiting") }
    return try body()
}
```

An `isolated` parameter makes the function's isolation *be* the argument: for the duration of the call, the function is inside that room, and may synchronously touch that room's state.

**`#isolation`** is the macro that captures *the caller's* isolation as a default argument:

```
func traceStep(
    _ message: String,
```

```
    isolation: isolated (any Actor)? = #isolation   // inherit caller's room
) async {
    // Runs in the caller's room: no hop, no door, non-Sendable
    // arguments flow freely because no boundary is crossed.
}
```

**Why this matters architecturally.** A plain `async` helper that a `@MainActor` caller invokes may (in pre-6.2 semantics) hop *away* to the hallway crew, forcing every argument to cross a boundary—suddenly you need everything `Sendable` for what is conceptually a local call. Inheriting the caller's isolation via `#isolation` (or the newer `nonisolated(nonsending)`, 4.8) makes the helper a polite guest instead of a foreign call: no crossing, no `Sendable` tax, in direct service of *Annotations Are a Tax to Minimize* (2.33).

**Misuse**—accepting an actor parameter *without* `isolated` and expecting sync access:

```
func poke(_ counter: Counter) {
    counter.value += 1     // 🚫 nonisolated context; you hold an address,
not a key
}
func poke(_ counter: isolated Counter) {
    counter.value += 1     // ✅ you are inside the room
}
```

**Neighbors.** `nonisolated(nonsending)` (4.8) is this pattern made into a calling convention; `@isolated(any)` (4.12) is the same idea for function *values*.

### 4.8 `nonisolated(nonsending)` and `@concurrent`

These two settle the oldest ambiguity in the model: *where does a nonisolated `async` function actually run?* History gave two contradictory answers—2022 said "always on the concurrent pool" (SE-0338); 2025 reversed it to "in the caller's room" (SE-0461), with these two spellings making the choice explicit. (The reversal itself, and the holes it caused, are a Part 6 case study; see Appendix C.)

**`nonisolated(nonsending)`**—run *in the caller's room*. No boundary is crossed, so non-Sendable arguments flow freely:

```
final class Draft {          // deliberately non-Sendable, mutable
    var text: String = ""
}

nonisolated(nonsending)
func spellcheck(_ draft: Draft) async -> [Issue] {
    // Runs as a guest in the caller's room. `draft` never crossed
```

```
    // a boundary, so no Sendable requirement, no box transfer.
    ...
}
```

**`@concurrent`**—always hop to the hallway crew. Now every argument *must* cross:

```
@concurrent
func render(_ scene: sending Scene) async -> Image {
    // Runs on the global concurrent executor, in parallel with the caller.
    // `scene` had to be a surrendered box (2.15) to get here.
    ...
}
```

**How to choose.** Ask one question: *is this function's job to run in parallel with its caller?* If yes—it's CPU work you want off the front desk—say `@concurrent`, and accept the crossing obligations as the honest price of parallelism. If no—it's just async because it awaits things—prefer the caller's room, and inherit isolation. Parallelism should be a decision, not a side effect of the word `async` (2.32).

**Errors you'll meet.** Adopting `@concurrent` on a previously caller-isolated function makes previously-fine call sites erupt with `sending 'x' risks causing data races`. That's not noise; that's the crossing tax you just opted into. Either the arguments become memos (`Sendable`), or surrendered boxes (`sending`), or the function doesn't actually need to be parallel.

**Neighbors.** The Approachable Concurrency flag bundle (4.10) makes `nonisolated(nonsending)` the default for nonisolated async functions, module-wide.

### 4.9 Actor lifecycle: `isolated deinit` and flow-sensitive initializers

Two features that uphold *No Homeless Code* (2.11) at the awkward ends of an actor's life.

**Initializers.** Before an actor's `self` is fully formed and shared, there is no receptionist yet—so a nonisolated `init` may freely set stored properties *until* `self` escapes (is captured, passed, or used as a whole); after that, normal room rules apply. The compiler tracks this flow-sensitively. If you see `error: actor 'self' can only be captured by a closure from an async initializer`, you've tried to share the room's address before the walls were up: finish construction first, or move the escaping work into a method called after init.

**`isolated deinit`.** A deinitializer ordinarily runs wherever the last reference dies—*any* room, any thread. Marking it `isolated` guarantees it runs on the

actor's own executor (enqueued if necessary), so teardown logic may touch room state safely:

```
@MainActor
final class VideoSession {
    isolated deinit {
        stopPlayback()        // main-actor state touched safely at teardown
    }
}
```

Use it when teardown must touch isolated state; skip it when `deinit` only releases references, since enqueueing has a cost.

### 4.10 Default isolation and the Approachable Concurrency bundle

**What it's for.** *Simple Programs Stay Simple* (2.32), delivered as a module setting. With default isolation set to `MainActor`, every unannotated declaration in the module is front-desk code. A single-threaded app is then simply *true by declaration*: everything is in one room, nothing crosses, no diagnostics—until you explicitly introduce parallelism with `@concurrent` or `Task.detached`.

```
// With default isolation = MainActor, this whole file is front-desk code.
// No annotations, no Sendable errors, no hops. (2.32 in action.)

var appState = AppState()          // a main-actor global: fine

func handleTap() {                 // implicitly @MainActor
    appState.counter += 1
}
```

The **Approachable Concurrency** build setting bundles this with the flags that make the modern semantics whole—caller-isolated nonisolated-async by default (4.8), Sendable inference for methods and key paths, usability relaxations for global-actor-isolated types, and isolated-conformance inference.

**The price** is *Dialects Are the Price* (2.36): the same source file means different things under different settings. Inside one module you'll never notice. Across modules—and in code you paste from the internet, written under someone else's dialect—you will. State your module's dialect in its README; when filing compiler bugs, state it in the report. A Swift file no longer has a meaning; a (file, mode, flags, default-isolation) tuple does.

**Misuse**—enabling MainActor-by-default on a *server* or compute-heavy module. The default exists for programs that are conceptually single-threaded. If your module's job is parallel throughput, a front-desk default just funnels everything

through one receptionist; leave it `nonisolated` and annotate the few genuinely main-actor bits.

### 4.11 `Sendable`, `@Sendable`, and `@unchecked Sendable`

**What `Sendable` is for.** The memo stamp: a marker conformance certifying that copies of this value may travel to any room without creating shared mutable state—*Copies Travel Free* (2.13). The compiler checks it structurally, which is why most of your types get it for free:

```
struct Order: Sendable {          // ✅ all stored properties Sendable ⇒ OK
    let id: OrderID
    let items: [LineItem]         // Array of Sendable is Sendable
    let total: Decimal
}

final class Invoice: Sendable { // ✅ final + all lets of Sendable type ⇒ OK
    let order: Order
    let issuedAt: Date
    init(order: Order, issuedAt: Date) { ... }
}
```

**Misuse**—claiming the stamp for something with shared mutable guts:

```
final class Basket: Sendable {
    var items: [LineItem] = []
    // 🚫 error: stored property 'items' of 'Sendable'-conforming class
    //    'Basket' is mutable
}
```

The compiler is right: two rooms holding references to one mutable `Basket` is *exactly* a data race waiting to happen. Your options, in order of preference: make it a `struct` (copies genuinely separate); make the class immutable; give it to an actor to own; or protect it internally and swear the oath—

**`@unchecked Sendable`** (oath 2.37-a): "thread-safe by means the checker cannot see."

```
final class Basket: @unchecked Sendable {
    private let lock = NSLock()
    private var items: [LineItem] = []       // every access below takes
`lock`

    func add(_ item: LineItem) { lock.withLock { items.append(item) } }
    var snapshot: [LineItem] { lock.withLock { items } }
}
```

The oath is only as good as the discipline: *every* access through the lock, no exceptions, forever, including the ones your successor adds next year. This is precisely the discipline the checker was built to replace, so treat `@unchecked Sendable` as a border-town tool (wrapping legacy code), not a lifestyle. `Mutex` (4.20) gives you the same shape *with* checking.

**`@Sendable` on closures** is the same stamp for function values: it may capture only Sendable values, and only by value. This is what makes a closure safe to run from another room:

```
var hits = 0
let handler: @Sendable () -> Void = {
    hits += 1
    // 🚫 error: mutation of captured var 'hits' in concurrently-executing
code
}
```

The error is the design working: a closure that mutates its captures is a cable back into your room; stamp it "travels freely" and you've mailed someone your wall socket. Capture a snapshot instead (`[hits]`), or route the mutation through an actor.

**Errors you'll meet:**

```
error: type 'Basket' does not conform to the 'Sendable' protocol
note: class 'Basket' does not conform to the 'Sendable' protocol
```

Something non-Sendable is trying to cross a room boundary—a capture in a `Task`, an argument to a `@concurrent` function, an actor method return. First question: *should* it cross? Often the right fix is architectural (let the room keep its machinery and send memos out). If it should cross, either earn the stamp or move it as a box (4.12).

```
warning: capture of 'self' with non-Sendable type 'Controller' in a
         '@Sendable' closure
```

Usually a `Task { }` or completion handler capturing a class. Ask which room `self` belongs in; if it's `@MainActor`, make the closure `@MainActor` too and the capture becomes legal—same room, no crossing.

**Architecture.** `Sendable` is not an annotation you add; it is a property your design either has or lacks. Systems built from value-type messages and actor-owned machinery are Sendable-clean by construction. Systems built from webs of

mutable classes fight the checker forever—the checker is the messenger, not the problem.

**Neighbors.** `sending` (4.12) moves what can't be stamped; `SendableMetatype` (below) is the stamp for *types themselves*; `~Sendable` (4.6) declines the stamp.

**`SendableMetatype`** is a niche but principled corner: it marks that a *type's identity* (its metatype, its conformances) may cross rooms. Its real job is to fence isolated conformances (4.12): generic code that could carry a conformance out of its room must require `SendableMetatype`, and front-desk conformances refuse to satisfy it. You'll meet it as the reason a generic function can't accept your main-actor-conforming type—which is the fence doing its job (2.12).

### 4.12 Regions, `sending`, `@isolated(any)`, and isolated conformances

This section is the heart of the modern system: how non-Sendable things move.

**Regions: the box rule.** The compiler groups non-Sendable values into *regions*: if one value can reach or alias another, they share a region and move together—*Move the Whole Box* (2.14). You never declare regions; a flow-sensitive analysis infers them per function body. What you experience is the *freedom* it buys:

```
func makeAndShip() async {
    let report = Report()              // non-Sendable, fresh box
    report.add(section: intro)         // still ours; build it up freely
    await archive.store(report)        // box handed into the archive room ✅
    // report and everything it references is now the archive's.
}
```

That compiles *without* `Report` being Sendable, because at the crossing the box was provably disconnected—nothing on our side still cables into it. The moment that stops being true, the checker stops you:

```
func makeAndLeak() async {
    let report = Report()
    await archive.store(report)        // box handed over…
    report.add(section: appendix)
    // 🚫 error: 'report' used after being sent to actor 'archive'
    //    …you kept no key (2.14)
}
```

And aliasing is what makes it a *box*, not a value:

```
let report = Report()
let alias = report                     // same region—cabled together
await archive.store(report)
```

```
print(alias.pageCount)
// 🚫 error: 'alias' cannot be used after 'report' is sent—same box
```

**`sending`: the signed hand-off.** Region inference works within one function body; *Check Locally, Never Globally* (2.34) forbids it from peeking across calls. So when a *signature* needs to express "give me a disconnected box," you write `sending`—*Hand It Over and Let Go* (2.15):

```
func enqueue(_ job: sending Job) {          // caller must surrender the box
    Task.detached { job.run() }            // callee may re-send it anywhere
}

func makeJob() -> sending Job {            // result arrives as a fresh box
    Job()                                  // caller may send it onward
}
```

Two clauses of the contract deserve emphasis, because each is a common misreading:

*The caller truly loses the value.* `sending` is not "trust me"; the region analysis verifies disconnection at every call site, and use-after-send is an error.

*The callee truly gains everything*—including the right to send the box onward to a third room. This is the clause the motivating bug of this document violated (Appendix C): other parts of the compiler assumed "same room on both ends of a call ⇒ nothing really moved," which `sending`'s re-send right makes false. When you *write* APIs taking `sending` parameters, honor the symmetric reading: you have been given a box, free and clear, and what you do with it is your business—which is exactly why the caller had to fully let go.

**Errors you'll meet, and how to read them:**

```
error: sending 'model' risks causing data races
note: 'model' is captured by a main actor-isolated closure. Later accesses
      could race
```

The box wasn't disconnected: something else still cables into `model`. Find the cable—often an innocent-looking local alias, a capture in an earlier closure, or storage in `self`. Fixes, best first: (1) create the value *just before* sending so nothing else references it; (2) send a Sendable snapshot instead; (3) restructure so the receiving room creates the value itself.

```
error: returning 'cache' risks causing data races
note: actor-isolated 'cache' cannot be returned as a 'sending' result
```

You promised a fresh box but tried to hand out room furniture. Rooms don't give things back (2.14); return a copy.

**`@isolated(any)`: the labeled box.** Every other function conversion in Swift can *erase* isolation from a type—the shipping label falls off, and only the checker's regional bookkeeping remembers where the contents belong. `@isolated(any)` is the opposite: a function type that *dynamically carries* its isolation, inspectable at runtime via its `.isolation` property:

```
func schedule(_ work: @escaping @isolated(any) () -> Void) {
    if work.isolation === MainActor.shared {
        runOnNextFrame(work)          // we can READ the label and route
smartly
    } else {
        runOnPool(work)
    }
}
```

This is the only feature in the language that upholds *Labels Never Lie* (2.17) by construction—the label physically travels with the value. It is how `Task { }` inherits your actor context (2.21): the initializer takes an `@isolated(any)` closure and reads the label. When you write scheduling primitives, executors, or anything that stores heterogeneous closures for later, `@isolated(any)` is the correct parameter type—it preserves the one fact everything else depends on. (A long-term direction for the language, discussed in Part 6, is to make *all* function values this honest.)

**Isolated conformances.** A conformance can carry a room label too (2.12): `extension Chart: @MainActor Drawable` says the conformance itself is front-desk property—usable only where main-actor isolation is guaranteed:

```
@MainActor final class Chart {}
protocol Drawable { func draw() }
extension Chart: @MainActor Drawable {    // conformance lives at the front
desk
    func draw() { ... }                   // may touch main-actor state ✅
}

func renderLater<T: Drawable>(_ item: T) where T: SendableMetatype {
    Task.detached { item.draw() }         // could smuggle the conformance
out…
}
renderLater(chart)
// 🚫 error: main actor-isolated conformance of 'Chart' to 'Drawable' cannot
//    satisfy conformance requirement for a 'SendableMetatype' type parameter
```

The `SendableMetatype` fence is what keeps generic code from carrying a room-bound conformance into the hallway. Before this feature, your choices were dishonest (`nonisolated` methods that secretly assumed the main thread) or painful; now the conformance can simply tell the truth.

### 4.13 `Task { }` and `Task.detached { }`

**What `Task { }` is for.** Hiring a courier with no parent—an *unstructured* task—while keeping the household connection (2.21): it inherits your priority, your task-local values, and your actor context. It is the bridge from synchronous code (a button handler, a delegate callback) into the async world:

```
@MainActor
final class SearchController {
    var results: [Result] = []

    func searchButtonTapped(query: String) {     // synchronous UI entry
point
        Task {                                   // inherits @MainActor
context
            results = await searchService.search(query)
            // `results` mutation is legal: this closure is main-actor code
        }
    }
}
```

Note why the mutation compiles: `Task { }`'s closure is `@isolated(any)` (4.12), so it *knows* it was created at the front desk and runs there.

**What `Task.detached { }` is for.** Hiring a stranger: no priority, no task-locals, no actor context. Detachment is occasionally right—genuinely independent background work whose lifecycle and context must not tie to the current one—and mostly wrong:

```
// 🪳 The detached-task reflex: "get me off the main thread!"
@MainActor func processImage() {
    Task.detached {
        let output = expensiveFilter(self.image)
        // 🚫 error: capture of 'self' with non-Sendable type...—and now you
        //    also lost priority and task-locals for no reason
        await MainActor.run { self.display(output) }
    }
}

// ✅ Say what you actually mean: parallel work, structured hand-off
@MainActor func processImage() async {
    let input = image                                  // Sendable snapshot out
```

```
    let output = await filterOffMain(input)          // @concurrent function
    display(output)                                  // back at the desk
(2.10)
}
```

**The real cost of unstructured tasks** is what they *escape*: `Children Finish First` (2.18) no longer applies. Nothing awaits them, nothing cancels them when their reason for existing disappears, errors evaporate unless stored. Every `Task { }` should have an answer to three questions: *Who awaits or stores this? What cancels it? Where do its errors go?* If the answers are "nobody, nothing, nowhere," you have a leak with scheduling privileges. Store the handle and cancel it when its owner goes away:

```
@MainActor final class LiveFeed {
    private var streamTask: Task<Void, Never>?

    func start() {
        streamTask = Task { for await event in feed { render(event) } }
    }
    func stop() { streamTask?.cancel() }     // an owner, an off-switch
}
```

(Since Swift 6.4, a `Task` whose failure type is a real error warns when you silently discard the handle—the compiler asking you question three.)

**Neighbors.** `async let` (4.2) and groups (4.14) are the structured alternatives—prefer them; `Task.immediate` (4.15) tweaks the start; cancellation (4.16) is the off-switch.

### 4.14 Task groups: `withTaskGroup`, `withThrowingTaskGroup`, `withDiscardingTaskGroup`

**What they're for.** Dynamic distribution with structure intact: hire *N* sub-couriers where *N* is a runtime number, collect their results as an async sequence, and let the scope guarantee that all of them finish or are cancelled before you leave (2.18).

```
func thumbnails(for urls: [URL]) async throws -> [URL: Thumbnail] {
    try await withThrowingTaskGroup(of: (URL, Thumbnail).self) { group in
        for url in urls {
            group.addTask { (url, try await makeThumbnail(url)) }
        }
        var out: [URL: Thumbnail] = [:]
        for try await (url, thumb) in group { out[url] = thumb }
        return out
    }
}
```

Everything the analogy promises is in that snippet: if one child throws, the siblings are cancelled and awaited; nothing outlives the closing brace; urgency flows into the children (2.20). Results arrive in *completion* order, not submission order (2.25)—carry the key with the result, as above, if identity matters.

**Misuse #1**—unbounded submission. `addTask` doesn't block; loop a million URLs and you've scheduled a million children. Structure gives you scoping, not backpressure. Meter it—the sliding-window idiom:

```
var iterator = urls.makeIterator()
for _ in 0..<maxConcurrent {                          // prime the window
    if let url = iterator.next() { group.addTask { try await fetch(url) } }
}
while let result = try await group.next() {           // one out, one in
    process(result)
    if let url = iterator.next() { group.addTask { try await fetch(url) } }
}
```

**Misuse #2**—escaping the group closure with `group` or with a child's box. The group is only meaningful inside its scope; the compiler enforces this with escaping and `sending` diagnostics. If you feel the urge to smuggle the group out, what you actually want is an `AsyncStream` (4.22) or an actor owning long-lived work.

**`withDiscardingTaskGroup`** is for children whose results nobody collects—the accept-loop of a server, fire-and-forget handlers. Children are cleaned up *eagerly* as they finish (a plain group holds results until you `next()` them—a slow-draining server accumulates them unboundedly), and one child's error still cancels the rest.

**Errors you'll meet.** `error: mutation of captured var 'out' in concurrently-executing code`—child closures run in parallel, so they can't all write into one local (2.13 applies to captures). Return values from children and merge them in the *parent's* loop, as above: the group is the funnel that turns parallel results back into one-at-a-time code.

### 4.15 `Task.immediate` and immediate child tasks

**What it's for.** A task that starts *synchronously, right now, in the calling context*, running until its first genuine suspension before yielding—instead of being enqueued for later. The isolation story is unchanged; only the start timing differs.

**When to use it.** Latency-critical starts: begin an animation this frame, issue the network request before returning from the event handler. If the code can complete without suspending, it completes before `Task.immediate` returns control—no scheduling gap.

**Misuse.** Reaching for it habitually. The default enqueue is fair and starvation-resistant; `immediate` borrows the *caller's* time slice. It's a targeted tool for "the enqueue latency is the bug," not a general accelerator.

### 4.16 Cancellation: `isCancelled`, `checkCancellation`, `withTaskCancellationHandler`, and shields

**The model.** *Cancellation Is a Request, Not a Command* (2.19): a flag that flows down the family tree, and that work observes voluntarily. Nothing is preempted; nobody is killed mid-write; invariants never shatter because cancellation "happened to" you—you *choose* your exits.

```
func export(_ pages: [Page]) async throws -> ExportedDoc {
    var doc = ExportedDoc()
    for page in pages {
        try Task.checkCancellation()      // polite exit point: throws
CancellationError
        doc.append(try await render(page))
    }
    return doc
}
```

Three observation tools, three uses: `Task.isCancelled` (a quiet boolean—for winding down with partial results), `Task.checkCancellation()` (throw and unwind—the usual choice), and `withTaskCancellationHandler`—the one that solves the *waiting* problem. A task blocked at a door can't poll a flag; the handler runs immediately, from wherever cancel was called, letting you unblock the wait:

```
func fetch(_ request: Request) async throws -> Response {
    let handle = connection.begin(request)
    return try await withTaskCancellationHandler {
        try await handle.response          // may wait a long time…
    } onCancel: {
        handle.abort()                     // …this runs promptly and frees
it
    }
}
```

Mind that the handler runs *concurrently* with the body, from another context—it must touch only Sendable/thread-safe things (the checker will hold you to it).

**Misuse**—ignoring cancellation in long loops. Cancellation is cooperative; code that never observes the flag is *uncancellable*, and every timeout or "user navigated away" upstream of you silently stops working. Long loops check each iteration; long waits get handlers.

**`withTaskCancellationShield`** masks *observation* of the flag inside a scope—for cleanup that must finish even in a cancelled task (releasing a distributed lock, committing a journal). The flag itself stays raised (2.19); you're covering your own eyes, briefly and on purpose, not lowering the flag. Shield the smallest possible scope.

### 4.17 `@TaskLocal`

**What it's for.** Values that follow the errand rather than the room or the thread—request IDs, trace contexts, deadlines. A task-local is *bound for a scope* and inherited by every child in the subtree (2.21):

```
enum Tracing { @TaskLocal static var requestID: String? }

func handle(_ request: Request) async {
    await Tracing.$requestID.withValue(request.id) {
        await validate(request)      // sees the ID
        await process(request)       // children and async lets see it too
    }
}                                    // binding gone; nothing to clean up
```

**Why the scoped shape.** Thread-locals broke in the task world because a task hops threads at every suspension (2.31). Task-locals follow the *logical* flow of work—the family tree—which is the thing that actually has continuity. The scoped `withValue` API means bindings can't leak: structure, again, doing the bookkeeping.

**Misuse**—using task-locals as ambient mutable state or hidden parameters that logic *depends* on. They're invisible in signatures; code that behaves differently because of one is code whose behavior you can't predict from its type. Diagnostics and context: yes. Control flow: pass a parameter.

**A boundary to know:** `Task.detached` deliberately inherits none (2.21)—a detached task starts with every task-local reset to default. If work needs the context, that's a hint it shouldn't be detached.

### 4.18 Custom executors: `SerialExecutor` and `TaskExecutor`

**What `SerialExecutor` is for.** Replacing a room's receptionist. An actor with a custom `unownedExecutor` runs all its work wherever you say—a specific dispatch queue, a dedicated thread, an I/O event loop:

```
actor Renderer {
    private let queue = DispatchSerialQueue(label: "renderer")
    nonisolated var unownedExecutor: UnownedSerialExecutor {
        queue.asUnownedSerialExecutor()      // DispatchSerialQueue conforms
    }
}
```

Now `await renderer.draw()` executes on that queue—and, crucially, code that checks isolation *dynamically* (2.26) agrees: the actor world and the queue world share one notion of "who's in the room," which is the whole trick for coexisting with GCD-era code (Part 5).

**The oath.** Conforming to `SerialExecutor` is honor-system item 2.37-f: *you* now guarantee *One Visitor at a Time* (2.8)—mutual exclusion and total order. The compiler builds the entire safety proof for that room on your word. An executor that ever runs two jobs concurrently, or reorders them into overlap, is a data-race generator wearing a safety badge. Test it accordingly.

**What `TaskExecutor` is for.** A milder tool: a *preference* for where a task's roomless (nonisolated) code runs—e.g., keep this subsystem's hallway work on its own thread pool. It never overrides isolation: room code still runs in its room (2.10). Use it for performance shaping (thread affinity, avoiding oversubscription), never for correctness—correctness belongs to isolation.

**When to reach for either.** Late, honestly. The defaults are good. Custom executors earn their keep at real boundaries: an event-loop library, a render thread mandated by a graphics API, a legacy queue whose ordering other code depends on.

### 4.19 Dynamic isolation tools: `assumeIsolated`, `preconditionIsolated`, `assertIsolated`

**What they're for.** *You May Always Ask Where You Are* (2.26), as API. These are the border-crossing papers for code that *is* in a room but can't prove it statically—legacy callbacks, C callbacks, delegate methods from frameworks that promise "called on the main thread" in documentation instead of types:

```
// Legacy framework: "delegate methods are always called on the main thread."
func legacyDelegateCallback(_ info: Info) {        // nonisolated, sync
```

```
    MainActor.assumeIsolated {                          // dynamic proof → static
license
        viewModel.apply(info)                           // front-desk access,
checked ✅
    }                                                   // traps if promise was
false
}
```

`assumeIsolated` converts a runtime check into static isolation for a scope—and *traps* if the assumption is false. That trap is a feature (2.30): the legacy framework's documentation just failed an audit, deterministically, at the exact line where the promise broke—instead of corrupting memory quietly. `preconditionIsolated()` and `assertIsolated()` are the check without the license: executable documentation for "this must only be called from room D."

**Misuse**—using `assumeIsolated` to *silence* the checker rather than encode a real external guarantee:

```
// 🍾 "It crashed, so I assumed harder"
nonisolated func onTimer() {
    MainActor.assumeIsolated { model.tick() }   // timer fires on a
background
                                                 // thread ⇒ trap, every
time
}
```

The tool asserts a fact; it doesn't create one. If nothing outside the program guarantees you're at the front desk, you aren't—hop properly (`await MainActor.run { ... }` or a `Task { @MainActor in ... }`).

**Architecture.** Every `assumeIsolated` marks a spot where a real-world contract enters the typed world. Keep them at the perimeter—thin adapter shims around legacy entry points—so the interior of your program holds only static proofs.

### 4.20 Mutex and Atomic

**What they're for.** Synchronization memory—Part 3's fourth kind. Sometimes an actor is too much room for two integers, and the door tax (`await` on every access) too high. `Mutex<State>` is a lock the compiler *understands*:

```
final class Metrics: Sendable {                     // genuinely Sendable—
checked!
    private let counts = Mutex<[String: Int]>([:])

    func record(_ event: String) {
        counts.withLock { $0[event, default: 0] += 1 }
```

```
    }
}
```

Two things distinguish this from `NSLock`-plus-discipline: the protected state lives *inside* the `Mutex`, unreachable except through `withLock` (the cable can't escape), so the `Sendable` conformance above is checked, not sworn (contrast `@unchecked Sendable`, 4.11).

**The one iron rule:** no `await` inside `withLock`. A suspension while holding a lock walks out the door with the room key in your pocket (2.9 meets 2.23)—the API makes it hard by design (the closure isn't async), so don't get creative. Critical sections stay short, bounded, and synchronous; that's what keeps lock-holding compatible with *Always Keep Moving* (2.23).

**`Atomic`** is finer-grained still—single values, explicit memory orderings, the primitives of *Everything Needs an Order* (2.2) made available directly. Counters, flags, sequence numbers on hot paths. If you're not sure you need memory orderings, you don't; use `Mutex`.

**Choosing among actor / Mutex / Atomic:** an actor when the state has *behavior* (multi-step transactions, awaits inside its logic, identity in the design); a `Mutex` when it's small, hot, and synchronous; an `Atomic` when it's one machine word and you can name the ordering you need. All three end in the same place—every access ordered (2.2)—by different budgets.

#### 4.21 Continuations: `withCheckedContinuation` and `withUnsafeContinuation`

**What they're for.** The narrow bridge between callback-world and task-world. A continuation reifies "the rest of my task" as a value you hand to legacy code, which redeems it—*exactly once* (2.22):

```
func fetchUser(id: UserID) async throws -> User {
    try await withCheckedContinuation { continuation in
        legacyClient.fetchUser(id: id) { user, error in
            if let user { continuation.resume(returning: user) }
            else { continuation.resume(throwing: error ?? Unknown()) }
        }
    }
}
```

While the task waits, its crew thread is *freed* (2.24)—this is a suspension, not a block, which is why wrapping callbacks in continuations is always safe for the pool where semaphores never are (2.23).

**Misuse—the two cardinal sins**, both violations of *Resume Exactly Once* (2.22):

```
legacyClient.fetch { result in
    if let value = result.value { continuation.resume(returning: value) }
    if result.isCached { continuation.resume(returning: result.value!) }
    // 🍾 double-resume: checked ⇒ deterministic trap
    //    CHECKED CONTINUATION MISUSE: tried to resume its continuation
    //    more than once; unsafe ⇒ undefined behavior
}
// …and the quieter sin: an error path that returns without resuming at all.
// Checked ⇒ runtime warning: leaked its continuation—a courier waiting
// forever at a door that will never open.
```

Audit every path through the callback: every branch, every early return, every error case resumes, exactly once. `withUnsafeContinuation` is the identical API minus the enforcement (oath 2.37-c)—measure before you take that oath; the checking is cheap and the failure it prevents is a permanently hung task.

**Architecture.** Continuations are single-shot. The moment a callback can fire more than once—progress handlers, event listeners—you want `AsyncStream` (4.22). And keep continuation shims at the perimeter, one per legacy API, named like the async function they wish they were.

### 4.22 `AsyncStream` and `AsyncThrowingStream`

**What they're for.** The multi-shot bridge: turning a callback that fires repeatedly into an `AsyncSequence` (4.3), with explicit buffering and cleanup:

```
func locations() -> AsyncStream<Location> {
    AsyncStream(bufferingPolicy: .bufferingNewest(1)) { continuation in
        let delegate = LocationDelegate { continuation.yield($0) }
        manager.startUpdating(delegate)
        continuation.onTermination = { _ in
            manager.stopUpdating()          // consumer left ⇒ turn sensors
off
        }
    }
}

for await location in locations() { updateMap(location) }
```

The two decisions the API forces on you are exactly the two that matter:

*Buffering policy*—producers and consumers run at different speeds; when the consumer lags, do old values pile up (`.unbounded`), fall off the front (`.bufferingOldest(n)`), or get replaced (`.bufferingNewest(n)`)? For live state like locations, newest-wins is honest: stale positions have no value. For a command

sequence, dropping *anything* is a bug—buffer unbounded and enforce backpressure upstream.

*Termination*—`onTermination` fires when the consumer stops (break, cancellation, deallocation), which is your one reliable hook for releasing the underlying resource. A stream without an `onTermination` that stops hardware is a resource leak in stream clothing.

**Misuse**—treating a stream as a broadcast. Each `AsyncStream` supports one consumer; two `for await` loops on the same stream split the values unpredictably. For distribution, look to `swift-async-algorithms`' sharing operators or have an actor own the source and vend fresh streams.

#### 4.23 `@preconcurrency` and `@available(*, noasync)`

**What `@preconcurrency` is for.** *Soft Borders During Migration* (2.29) as an annotation—a graded trust system for the boundary with unchecked code, from softest to firmest:

*On an import*—`@preconcurrency import OldSDK`: "this module hasn't adopted checking; don't punish me for its missing annotations." Sendable diagnostics traceable to that module are downgraded or suppressed. This is oath 2.37-e—trust *assumed*.

*On your own declarations*—lets you *add* concurrency annotations to a shipped API without breaking pre-strict clients: new truth for the migrated, old tolerance for the rest. Trust *grandfathered*.

*On a conformance*—`class Legacy: @preconcurrency Delegate`—for the classic bind: a `@MainActor` class conforming to a protocol whose requirements are nonisolated, where the framework really does always call on the main thread but its types don't say so. Static checking is waived, **and the compiler inserts a runtime isolation check at every entry point the conformance vends** (2.30). Trust *verified dynamically*: if the framework ever calls off-main, you get a clean trap at the boundary, not a race in your model.

That gradient—assumed → grandfathered → dynamically checked → statically proved—is the migration story in miniature, and each `@preconcurrency` you delete is a step up it. They are meant to be deleted: when a dependency adopts checking, a since-fixed `@preconcurrency import` starts warning that it's unused. Heed it; each one you remove hands another border to the compiler.

**What `noasync` is for.** *Thread-Bound Code Stays Put* (2.31). An API whose correctness depends on staying on one thread—`pthread_mutex_lock`'s unlock-on-the-same-thread rule, thread-local storage—is a landmine in a world where tasks hop threads at every `await`. Marking it `@available(*, noasync)` makes it uncallable from async contexts:

```
@available(*, noasync, message: "Thread-affine; use Mutex, or call from a dedicated thread")
func enterLegacyCriticalSection() { ... }
```

If you must use such an API from the async world, give it a dedicated thread or a custom executor (4.18) that pins the work, and vend an async facade.

#### 4.24 Clocks and time: `Clock`, `Instant`, `Duration`

The time substrate for scheduling: `try await clock.sleep(for: .seconds(2))` is a *suspension*—the crew thread is freed (2.24) and the sleep is cancellable like any wait (2.19), which is everything `Thread.sleep` isn't (it blocks a crew thread: a 2.23 violation). Injecting a `Clock` into time-dependent code is also the difference between tests that take seconds and tests that take milliseconds—a test clock is a room where time obeys you.

---

## Part 5: Living with the old world

Swift concurrency runs on the same kernel threads that `Thread` and Dispatch use—executors are an *organization* of those threads, not a replacement for them. Main-actor work runs on the main thread; the crew's work runs on a pool; a custom executor can be a dispatch queue wearing a receptionist's badge (4.18). The two worlds are one building with two generations of signage, and the rules of engagement follow from the requirements you already know.

**The translation table**, with the catch in each row:

| Old world | New world | The catch |
|---|---|---|
| Serial `DispatchQueue` | `actor` | An actor is not FIFO (2.25) and is re-entrant at awaits (2.9). A queue block is atomic start-to-finish; an actor method is transactional *per await-free segment*. |
| `DispatchQueue.main` | `@MainActor` | Exact correspondence, by construction—the main actor's |

| | | |
|---|---|---|
| | | executor drains the main queue. |
| Concurrent queue | The crew (global executor) | The crew never grows to cover blocked threads. GCD's thread-explosion valve is gone on purpose (2.23). |
| `DispatchGroup.wait` | Task group (4.14) | The structured version *awaits*; calling `group.wait()` on a crew thread is a 2.23 violation. |
| `DispatchSemaphore` across work items | **Nothing. Deliberately.** | Blocking a crew thread on a signal from another task can deadlock the whole pool, and semaphores are invisible to priority escalation (2.20). The canonical crew-rule violation. |
| `NSLock` / `os_unfair_lock` | `Mutex` (4.20) | Fine if critical sections are short and contain no `await`. Thread-*affine* locks (recursive locks, ownership asserts) break when tasks hop threads—that's what `noasync` quarantines (2.31). |
| `Thread.current`, thread-locals | `@TaskLocal` (4.17) | Thread identity is meaningless mid-task; context follows the errand tree instead (2.21). |
| `DispatchWorkItem.cancel` | `Task.cancel` (4.16) | Same philosophy—both cooperative, neither preempts (2.19). |
| QoS classes | `TaskPriority` | Escalation across awaits is built in (2.20); GCD needed manual plumbing and lost donation through semaphores. |

**Calling old from new.** The rule is the crew rule (2.23): a blocking legacy call must not squat on a pool thread beyond "brief and bounded." Wrap callback APIs in continuations (4.21) or streams (4.22)—those *suspend*, freeing the thread. For genuinely blocking work (a C library that computes for seconds), give it its own thread or a dedicated `TaskExecutor` (4.18); do not park the crew on it.

**Calling new from old.** Three doors in: `Task { }` from any queue context (fire-and-forget into the async world, 4.13); `MainActor.assumeIsolated` when the calling queue *is* the main queue (4.19); and a custom executor identity claim when an actor and a queue are secretly the same room (4.18), letting `dispatch_assert_queue`-style reasoning and actor reasoning agree.

**State shared across the border** must satisfy *Copies Travel Free* (2.13) the hard way: an actor both sides talk to, a `Mutex`, or `@unchecked Sendable` with a real lock (oath 2.37-a). Understand the asymmetry here, because it is why migration hurts: the checker cannot *see* GCD's queue discipline, so old code that is perfectly race-free by convention is *unverifiable*, not unsafe. Migration is the act of converting conventions into declarations—the safety was usually already there; you're making it visible.

**Callbacks crossing back** are presumed hostile (2.28): since SE-0463, every imported completion handler is `@Sendable`, because the platform may invoke it anywhere. Bridge them at the perimeter (4.21, 4.22) and let the interior stay statically proved.

---

## Part 6: The rules of stewardship

Everything above describes the system at rest. This Part is about the system *in motion*—and it exists because the serious failures of Swift concurrency have not come from weak design. The design is sound; Part 3 sketched its proof. They have come from *change*: a rule written in 2022, sound under 2022's premises, silently invalidated by a feature added in 2024, discovered as a data race by a user in 2026. The premise was recorded in a pull-request description; the feature's review had no reason to look there; each compiler layer believed the other owned the check. (Appendix C dissects the canonical example.)

A concurrency guarantee is a chain (Part 1), and the chain's real enemy is not the bad link but the *unrecorded assumption* between links. So these are rules about writing things down—for pitch authors, proposal authors, PR authors, and committers. They are cheap. Their absence has been expensive.

### The Ledger Rule (6.1)

**Every carve-out is registered, with its premise, in one greppable place.**

A *carve-out* is any rule that trades theoretical strictness for ergonomics in a specific pattern: "we allow this conversion because the value can't escape," "we skip this diagnostic because matching isolation means nothing moved." Carve-outs are legitimate—they are how *Annotations Are a Tax to Minimize* (2.33) coexists with the promise (2.3). But every carve-out is a standing debt: a proposition that must remain true as the language changes.

The rule: a normative `ConcurrencySemantics.md` lives in the compiler repository, containing every requirement of Part 2 (with these names and numbers, or better ones) and a **carve-out ledger**. Each ledger entry states: the rule, the exact soundness premise it relies on, the requirements it touches, the owning subsystem, and a link to its tests. A carve-out not in the ledger is, by definition, a bug—even if it's currently sound.

In code, suppression heuristics cite their ledger entry:

```
// Ledger CO-7: sound per Move the Whole Box (2.14) ONLY IF no `sending`
// operand is involved—see Hand It Over and Let Go (2.15), re-send right.
```

instead of a bare English sentence that no future search will ever connect to the feature that breaks it.

### The Crossing Audit Rule (6.2)

**Any pitch or proposal that adds or changes a way values cross isolation boundaries must include a "Crossing audit" section.**

Exactly as proposals today must address Source Compatibility and ABI Stability, a proposal touching crossing—new send mechanisms, new conversions, new inheritance of isolation, changes to where async functions run—must enumerate every ledger entry (6.1) whose premise mentions crossing, and state for each: *still sound*, or *invalidated, and here is the fix*. This single template change is the measure that would have caught the `sending` × conversion-rule collision at review time, before it shipped as a hole. The audit is mechanical *because* the ledger exists; that's the point of the ledger.

### The One Owner Rule (6.3)

**Every safety judgment has exactly one owning subsystem, named in writing. Other layers may duplicate the check only as assertions—never as the sole enforcement.**

The ownership assignment consistent with the current architecture (details in Appendix B): type-shaped, flow-insensitive facts—declaration isolation, Sendable conformance, conversion legality judged from types alone—belong to the type checker. Value-flow facts—box status, crossing legality, use-after-send—belong to the region analysis. The corollary does real work: **a type-checker rule whose soundness depends on a value-flow fact ("this value never later crosses") is illegitimate as written**—it must either be strengthened into a pure type judgment (refuse the conversion) or compiled into information the region analysis can enforce (6.5). "Which subsystem is responsible for preventing this bug?" must always have a one-line answer that predates the bug; when engineers have to ask it on the forums, the answer arrived years too late.

For PRs, the rule bites concretely: a PR fixing a safety issue states, in its description, which subsystem owns the violated judgment and why the fix lands there rather than in the neighbor. A fix that patches the *other* layer—adding a region-analysis special case for a type-checker leak, say—is a red flag that must be justified explicitly, because compensating patches are how unowned judgments accumulate.

### The Litmus Rule (6.4)

**Every requirement, every ledger entry, and every soundness fix is witnessed by executable litmus tests.**

Prose specifications rot; the memory-model community learned this decades ago and moved to litmus tests—tiny programs with pinned verdicts. The rule: a `test/Concurrency/litmus/` corpus where each test states a program, a dialect tuple (language mode × flags × default isolation—this is *Dialects Are the Price* (2.36) made checkable), and a verdict: *accepted*, *this diagnostic*, or *traps here*. Coverage is indexed by requirement name, so "which tests witness *Hand It Over and Let Go* (2.15)?" is a query, not a research project. Every hole fix adds its reproduction, permanently. A proposal changing semantics ships its litmus tests *in the proposal*, which incidentally makes review concrete: reviewers argue about verdicts on programs, the most productive argument there is.

### The No Silent Erasure Rule (6.5)

**No compiler stage may discard a value's isolation without leaving it recorded somewhere another stage can see.**

This is *Labels Never Lie* (2.17) turned into an enforceable invariant. For every function-typed value: either its type still carries its isolation, or it is `@isolated(any)` (label travels in the value, 4.12), or the region analysis records the value's region as belonging to that isolation. Concretely: an isolation-erasing conversion must produce a value whose region is *born room-bound*—after which no special cases are needed, because room-bound boxes already can't be sent (2.14). The invariant is assertable in the compiler: where the layers disagree about what got erased, that's an assertion failure at compile time instead of a data race at run time. Appendix C shows how this one rule mechanically subsumes the entire known hole family.

### The Model Rule (6.6)

**"This feature is not in the formal model" is a named review flag.**

The region calculus has a published soundness proof—for the calculus. Isolation-erasing conversions, task-isolated regions, `sending` results, isolated conformances: the features where holes actually appeared are precisely the ones the calculus doesn't model. Keeping a modest mechanized model in sync with the shipped surface is a student-project-sized effort, not a moonshot, and it converts "we believe this composes" into "we checked." Until a feature is modeled, its proposal says so, and reviewers weigh that.

### The Citation Rule (6.7)

**Commits and PRs touching concurrency semantics cite requirements by name and number.**

Not for ceremony—for *searchability in both directions*. When a future proposal changes what *Hand It Over and Let Go* (2.15) means, `git log --grep="2.15"` must surface every commit whose correctness leaned on the old meaning. The unrecorded-premise failure (Appendix C) was, at bottom, a search that couldn't succeed: the premise lived in a PR description, connected to nothing. Citations are how the codebase becomes greppable *by requirement*—which is what makes the Crossing Audit (6.2) cheap.

A commit message under this rule:

```
[Sema] Refuse isolation-dropping conversion for values that flow to
`sending` parameters

Upholds: Labels Never Lie (2.17), Hand It Over and Let Go (2.15)
Owner:   Sema (per One Owner Rule—conversion legality is type-shaped)
```

```
Ledger:  narrows CO-4; premise re-audited against SE-0430's re-send right
Litmus:  adds litmus/sending-erased-conversion-{1,2,3}.swift
```

Four lines of overhead; a permanent, queryable trace from code to promise.

### The templates, condensed

**A pitch** must state: which requirements (by name) it upholds, relies on, or modifies; its Crossing audit if it touches crossing (6.2); which subsystem owns each new judgment (6.3); and the premises it relies on that are *not* currently checked—the candidate ledger entries it would create.

**A proposal** adds: litmus tests with verdicts per affected dialect (6.4); the model-coverage statement (6.6); and, if any requirement's meaning changes, the explicit retraction or amendment—requirements are amended in writing or not at all. (The ghost of "and deadlocks," promised in 2020 and silently narrowed, is the cautionary tale: a requirements registry is also where *retractions* live, so promises can't fade—they must be kept or withdrawn.)

**A PR** states: owning subsystem (6.3), ledger entries touched (6.1), litmus tests added (6.4), requirement citations (6.7).

**A commit message** carries the citation block above.

The deepest fix, worth naming last because it is the *directional* one: the root enabler of the entire historical hole family is that Swift's ordinary function types don't carry isolation, so conversions must erase it—and erased facts must be heroically remembered elsewhere (6.5). The language already contains the alternative: `@isolated(any)`, the labeled box, where the fact simply travels with the value. A future language mode that deprecates isolation-erasing conversions—making `@isolated(any)` the ordinary currency of non-`@Sendable` function values, with migration per *Every Change Brings Its Own Movers* (2.35)—would convert *Labels Never Lie* (2.17) from an invariant needing policing into a triviality. Information that is never lost does not need to be guarded. That is, in the end, the design lesson of the whole affair: the cheapest invariant to enforce is the one you make unrepresentable to violate.

---

## Appendix A: Why this document exists

**The motivating incident.** In July 2026, an engineer posted a small program to the Swift forums under the title Which subsystem is responsible for preventing

this concurrency bug? The program used no unsafe opt-outs, compiled cleanly in Swift 6 mode, and raced—Thread Sanitizer confirmed it. The question in the title was the remarkable part: not "is this a bug" (it plainly was) but "*whose* bug is it"—and neither that thread nor its more detailed sibling (thread 87519) received any reply. Fix PRs for the underlying hole family sat unreviewed for months, review requests pending to the owners of two different compiler layers—because deciding *between* the layers requires an ownership doctrine that no document states. An engineer asking "fix Sema? fix RBI? both?" in a forum thread is an engineer asking for a requirements document in real time.

**Prior art: why no such document already existed.** What the ecosystem offers is a set of partial views, each valuable, none sufficient:

| Artifact | What it is | What it is not |
|---|---|---|
| TSPL "Concurrency" chapter | Narrative tutorial | Not normative; no requirements, no enforcement model |
| Swift Concurrency Proposal Index (Quinn, Apple DTS) | The best proposal inventory | A list, not a semantics; stops at Swift 6.2 |
| Swift Concurrency Roadmap (2020) | The original goal statement | Predates nearly everything; plans changed |
| Approachable data-race safety vision (2025) | Best statement of the character requirements (2.32–2.36) | Directional; explicitly not a specification |
| SE-0414 + appendix | The only genuinely formal fragment (the region dataflow) | One subsystem; silent on its interface to the type checker |
| Swift 6 migration guide | Operational how-to | Not a semantics |
| Massicotte's concurrency glossary | Community vocabulary | Definitions, not requirements |
| Swift Concurrency Manifesto (Lattner, 2017) | Historical ancestor | Substantially superseded |

No artifact states the requirements *as requirements*, maps features onto them, assigns each check an owner, or asks whether the set is consistent. The absence

of the third item—an owner per check—is precisely what thread 88002 was asking about.

**What the analysis found**, in brief (details in Appendices B and C): the requirement set is *consistent*—a mathematical model exists, adapted from a published, proved-sound type system, and nothing Swift promises is contradictory. The shipped implementation, however, is not currently a model of the requirements: at least nine acknowledged soundness holes existed as of July 2026, and they are not scattered—almost all cluster at a single architectural touchpoint, the unwritten hand-off contract between the type checker and the region analysis. And the three character requirements that shape everything—soundness, minimal annotations, modular checking—are jointly satisfiable, but only barely; the tension between them is exactly where the carve-outs and heuristics live, and the carve-outs are where the holes are. Hence Part 6: the holes are the interest payments on missing bookkeeping, and the bookkeeping is cheap.

**A note on the deadlock promise.** The 2020 roadmap promised to eliminate "data races *and deadlocks*." What shipped eliminates actor-mailbox deadlocks (via re-entrancy, 2.5) but permits crew-blocking deadlocks (2.23 violations) and classic lock-ordering deadlocks. The narrowing was reasonable; the *silence* of it is the instructive part—there was no registry in which to record the retraction, so the strong promise still circulates. A requirements document is also where promises go to be honestly amended.

---

## Appendix B: Who enforces what

The promise (2.3) is not enforced by one checker but by a pipeline, each stage seeing a different representation of the program and owning different judgments. The touchpoints between stages are where the trouble has lived, so here is the whole machine.

**The pipeline:**

1. **Sema**—the AST-level type checker (`lib/Sema/TypeCheckConcurrency.cpp`). Flow-*insensitive*: it judges declarations, expressions, and types, not what happens to a value across statements. Owns: isolation inference and the room-access rule (2.6, 2.7); Sendable conformance and capture checking (2.13); the legality of function-type conversions—including the historical carve-out permitting global-actor-dropping conversions; `@preconcurrency` diagnostic downgrades (2.29).

2. **SILGen**—lowering to SIL. Owns: insertion of `hop_to_executor` at isolated-function entries and at every resumption, which is *Run Where the Label Says* (2.10) made physical; and the dynamic isolation traps at unchecked boundaries (2.30).

3. **SIL mandatory passes**—above all **RBI**, region-based isolation (`TransferNonSendable.cpp`, `PartitionUtils.h`). Flow-*sensitive*: an optimistic forward dataflow over SIL. Owns: region tracking, crossing legality, use-after-send (2.14–2.16). Also here: flow-sensitive actor init/deinit isolation (2.11) and exclusivity diagnostics (2.1).

4. **Runtime.** Executor hops and priority escalation (2.20); the dynamic isolation predicates (2.26); checked-continuation exactly-once traps (2.22); dynamic exclusivity checks (2.1). The runtime is the last line of defense, and it has caught real failures of the layers above—the Swift 6.3.2 miscompile (missing hop back to the main actor after an await; #88993) surfaced in production as a `dispatch_assert_queue` trap, exactly as *Trap, Don't Race* (2.30) intends.

5. **Thread Sanitizer**—tooling, not guarantee: it detects at runtime what everything above missed.

**The ownership matrix** (the One Owner Rule (6.3), as the current architecture actually assigns it):

| Judgment | Owner | Backstop |
|---|---|---|
| "Declaration D lives in room ι" (2.6) | Sema | runtime assertions (2.26) |
| "This sync access to room state is legal" (2.7) | Sema | dynamic traps at unchecked boundaries (2.30) |
| "Type T is Sendable" (2.13) | Sema | none (marker protocol—no runtime presence) |
| "This function conversion is legal" | Sema | **supposed to be RBI—the disputed touchpoint; see Appendix C** |
| "Value v may cross boundary B" (2.14, 2.15) | RBI | none |
| "No use-after-send" (2.14) | RBI | none |
| "Code runs on its label's executor" (2.10) | SILGen + runtime hops | `preconditionIsolated`, `dispatch_assert_queue` |

| | | |
|---|---|---|
| "Actor init/deinit phases" (2.11) | SIL mandatory | — |
| "Exclusivity" (2.1) | Sema + SIL + runtime | runtime traps |
| "Continuations resume exactly once" (2.22) | Runtime (checked) | oath 2.37-c if unsafe |
| "A custom executor really serializes" (2.8) | **nobody—oath 2.37-f** | `checkIsolation`, if implemented |
| "Forward progress on the pool" (2.23) | **nobody—documented contract only** | a deadlocked app, in practice |

The two "nobody" rows and the one disputed row are precisely where the system's failures have concentrated. That is not a coincidence; it is what unowned judgments do.

**The formal fragment.** The one rigorously specified piece is RBI's dataflow, from SE-0414's appendix: program values partition into regions maintained as an alias graph; control-flow merge unions graphs (pessimistic toward connectedness, per *When Unsure, Assume Tangled* (2.16)); each instruction merges the regions of its non-Sendable operands and joins the result into the merged region; a send marks a region sent, and any later use of a sent region is diagnosed; the analysis is a standard optimistic forward dataflow over a finite lattice, so it converges. The theoretical basis—Milano, Turcotti & Myers, *A Flexible Type System for Fearless Concurrency* (PLDI 2022)—carries a soundness proof: in the calculus, well-typed programs are data-race-free. The catch, developed in Appendix C: the calculus does not model Swift's isolation-erasing function conversions, and the implementation added suppression heuristics the calculus does not license. The proof covers the core; the holes live in the unmodeled fringe—which is what the Model Rule (6.6) is about.

**Sema and RBI divide the labor on principle:** type-shaped judgments in the flow-insensitive layer, value-flow judgments in the flow-sensitive one. The division is sound. What was never written is the *contract across the touchpoint*—what Sema guarantees to RBI about isolation information surviving lowering, and vice versa. The strongest statements of that contract in existence are a 2022 pull-request description and inline comments in `PartitionUtils.h`. Part 6 exists because a load-bearing contract deserves better storage than that.

## Appendix C: The known holes and what they teach

As of July 2026, at least nine acknowledged soundness holes existed—programs with no opt-outs that compile in Swift 6 mode and race. This appendix catalogs them and dissects the canonical one, because the *pattern* matters more than any instance: nearly every hole is a violation of *Labels Never Lie* (2.17), and nearly every one sits at the Sema↔RBI touchpoint described in Appendix B.

**Anatomy of the motivating bug** (#90271, the subject of thread 88002; sibling of the original family report #79836):

```
@MainActor final class C { var state = 0 }

func send(_ fn: sending @escaping () -> Void) {
    Task.detached { fn() }          // legal: sending confers the re-send
right (2.15)
}

@MainActor func bug(_ c: C) {
    send { @MainActor in c.state += 1 }   // conversion drops @MainActor from
the type
    c.state += 1                          // races with the detached call
}
```

Walk the pipeline and watch the label die:

1. **Sema.** The closure's type is `@MainActor () -> Void`; the parameter wants `() -> Void`. A 2022 carve-out (from PR #62153—call it the *label-dropping conversion*) permits erasing the global actor when the context matches and the target is non-Sendable and synchronous. Its recorded justification, in the PR description: *"if we prevent the value from later leaving that isolation domain, it's OK to simply drop the global-actor."* In 2022 that premise was discharged by exhaustion—the only way for a function value to leave a domain was `@Sendable`, which the rule already refused. Sound, then.

2. **SE-0430 ships `sending` (2024).** Now there is a *second* way for values to leave a domain—one specifically designed to move non-Sendable values (2.15). The 2022 premise is silently invalidated. No artifact forces anyone to notice: the premise lives in a PR description, and *Hand It Over and Let Go*'s review had no checklist item that could have surfaced it. This is the failure the Crossing Audit Rule (6.2) exists to prevent.

3. **RBI.** The lowered closure still *behaves* main-actor-ish in SIL, and RBI has suppression heuristics of its own, also written in the pre-`sending` world—quoted from `PartitionUtils.h`: *"If our callee and region are both actor isolated and part of the same isolation domain, do not treat this as a send"*—the heuristic "matching isolation ⇒

nothing really moved," which `sending`'s re-send right makes false (`Task.detached { fn() }` moves it plenty). Both layers wave the program through; each believed the other owned the check (the One Owner Rule (6.3), violated in the wild).

4. **Runtime.** `Task.detached` runs the closure off the main actor, concurrently with `bug`'s own `c.state += 1`. Thread Sanitizer confirms the race the compiler promised away.

The exquisite detail: insert `let g = fn` before the send in some variants and RBI *catches it*—proving the touchpoint is implementable, and that the failure is one of contract, not capability. And the No Silent Erasure Rule (6.5) dissolves the whole family at once: if the label-dropping conversion produced a value whose region is *born main-actor-bound*, then sending it is already illegal under *Move the Whole Box* (2.14)—no special cases, no heuristics, the existing rules simply apply to preserved information.

**The catalog:**

| # | Reference | Shape | Touchpoint | Requirement violated | Status (July 2026) |
|---|---|---|---|---|---|
| H1 | #79836 | `@MainActor` closure → `() -> Void` conversion, passed to `sending` param | Sema carve-out + RBI both miss | 2.17 → 2.15 | Open; fix PR #86223 unreviewed since Dec 2025 |
| H2 | #90271 | Same family; TSan-confirmed; → thread 88002 | Sema | 2.17 | Open, triage |
| H3 | #82827 | Local `var` captured by `@MainActor` Task *and* `Task.detached`—no diagnostic | RBI | 2.14 | Open |
| H4 | #86896 | Non-Sendable captured in `Task { @MainActor in }`, then passed `@concurrent`; error computed but **suppressed** | RBI (squelch) | 2.14, 2.16 | Open, assigned |
| H | #89736 | Object with `@MainActor` | RBI (same- | 2.15, 2.12 | Open; fix PR |

| | | | | | |
|---|---|---|---|---|---|
| 5 | | conformance sent via `sending`; isolated-conformance method then called off-actor | isolation elision) | | #90075 unreviewed |
| H 6 | #76929 | `var` captured by `@MainActor` Task + mutated in nonisolated async fn | RBI | 2.14 | Acknowledged verbatim as "a data-race safety hole… a bug in region based isolation"; partially fixed in 6.1 |
| H 7 | #74820 | *Adding* `@MainActor` to a function removes a data-race error | RBI (isolated-caller suppression) | 2.14 | Closed by reporter; fix unverified |
| H 8 | forum 75332 / #65315, #71097 | Nonisolated async on non-Sendable class from `@MainActor`-inheriting Task: Swift 5.10 rejected, 6.0 raced—collateral of the SE-0338→SE-0461 reversal | RBI (task-inherited regions) | 2.14 | Mixed: partly fixed, partly by-design'd |
| H 9 | #88993 / #89214 | **Miscompile:** no hop back to MainActor after awaiting a `nonisolated(nonsending)` value with a `@concurrent` closure; production traps in Xcode 26.5 | SILGen/codegen | 2.10 | Fixed |

Reading the table: H1–H8 all live in the moving-things cluster (2.14–2.17) at or near the Sema↔RBI touchpoint. H9 is the lone *Run Where the Label Says* (2.10)

event—and the only one the runtime backstop caught by design, because 2.10 violations trap loudly (2.30) while 2.17 violations race silently. No hole is a counterexample to the theory of Part 3; every hole is a counterexample to the implementation's claim to implement it. That is simultaneously reassuring—nothing needs redesigning from scratch—and damning: the same unspecified touchpoint keeps producing the same failure, which is what unspecified touchpoints do.

One entry for symmetry: #75238 is a *false positive* (RBI bailing out on correct code)—a bug against *Annotations Are a Tax to Minimize* (2.33) rather than the promise (2.3). The two failure species are kept distinct on purpose: one costs comfort, the other costs the theorem.

The general shape, one more time, because it is Part 6's entire justification: **a premise sound in 2022, stored in an unqueryable place, invalidated by a 2024 change, discovered by users in 2025–26.** The Ledger (6.1) makes premises queryable; the Audit (6.2) makes changes check them; One Owner (6.3) makes someone answerable; Litmus (6.4) makes regressions loud; No Silent Erasure (6.5) removes the family's enabling mechanism; the Model Rule (6.6) catches the composition at design time; Citations (6.7) make all of it greppable.

---

### Appendix D: The proposal index

Every accepted or implemented Swift Evolution proposal governing concurrency, with the rule it contributes and the requirement(s) it serves. Verified against the canonical proposal headers and Quinn's index, July 2026. Ergonomics-only proposals are marked ·. "6.2ᶠ" = behind an upcoming flag in the Approachable Concurrency bundle.

| SE | Title | Ships | The rule | Requirement |
|---|---|---|---|---|
| 0176 | Exclusive Access to Memory | 4–5 | Law of Exclusivity | One Writer at a Time (2.1) |
| 0282 | Memory consistency model | 5.3 | C/C++ happens-before; racy access = UB | Everything Needs an Order (2.2) |
| 0296 | Async/await | 5.5 | `async` effect; suspension only at | The Room May Change |

| | | | | |
|---|---|---|---|---|
| | | | `await` | (2.9) |
| 0297 | ObjC interop | 5.5 | Completion-handler methods import as async | Old Code Speaks Through Translators (2.27) |
| 0298 | AsyncSequence | 5.5 | Pull iteration, suspension per element | (2.9) |
| 0300 | Continuations | 5.5 | Resume exactly once | Resume Exactly Once (2.22) |
| 0302 | Sendable | 5.5 | Crossability is a type property | Copies Travel Free (2.13) |
| 0304 | Structured concurrency | 5.5 | Task tree; scoped children; cooperative cancellation; priority | Children Finish First (2.18), Cancellation Is a Request (2.19), Urgency Flows Through (2.20) |
| 0306 | Actors | 5.5 | Isolated state; serialized access; re-entrancy at awaits | (2.6)–(2.9), No Deadlocks by Design (2.5) |
| 0311 | Task-locals | 5.5 | Tree-scoped values | Children Inherit the Household (2.21) |
| 0313 | Isolation control | 5.5 | `isolated` params; `nonisolated`; one isolation per decl | Everything Lives Somewhere (2.6) |

| | | | | |
|---|---|---|---|---|
| 0314 | AsyncStream | 5.5 | Buffered sync→async bridge | (2.22) generalized |
| 0316 | Global actors | 5.5 | Process-wide rooms; isolation inference | (2.6) |
| 0317 | async let | 5.5 | Implicit structured children | Children Finish First (2.18) |
| 0323 | Async main | 5.5 | Entry-point executor setup | (2.10) |
| 0327 | Actor init/deinit | 5.5+ | Flow-sensitive isolation around `self` escape | No Homeless Code (2.11) |
| 0329 | Clock/Instant/Duration | 5.7 | Time substrate · | — |
| 0331 | Pointers non-Sendable | 5.6 | Closes a crossing hole | (2.13) |
| 0336 | Distributed actor isolation | 5.7 | Isolation across processes (out of scope here) | (2.6) extended |
| 0337 | @preconcurrency | 5.6 | Graceful degradation at unchecked boundaries | Soft Borders (2.29) |
| 0338 | Nonisolated async execution | 5.7 | Off-actor by default— **reversed by SE-0461** | see (2.36) |
| 0340 | noasync | 5.7 | Quarantine thread-affine APIs | Thread-Bound Code Stays Put (2.31) |
| 0343 | Top-level concurrency | 5.7 | Top-level code is @MainActor | (2.6) |
| 0344 | Distributed actor runtime | 5.7 | Location-transparent messaging | — |
| 0374 | Clock.sleep(for:) | 5.9 | · | — |
| 0381 | DiscardingTaskGroup | 5.9 | Bounded-memory groups; auto error- | (2.18) |

| | | | | |
|---|---|---|---|---|
| | | | cancel | |
| 0388 | makeStream | 5.9 | · | — |
| 0392 | Custom executors | 5.9 | SerialExecutor contract; assumeIsolated | One Visitor at a Time (2.8), (2.26) |
| 0401 | Prune wrapper inference | 5.9/6[f] | Less isolation inference | (2.33) |
| 0410 | Atomics | 6.0 | SE-0282 reified | (2.2) |
| 0411 | Isolated default values | 5.10 | Defaults run in the declaration's room | No Homeless Code (2.11) |
| 0412 | Strict globals | 5.10 | Globals isolated / immutable / oath 2.37-b | (2.7) |
| 0414 | **Region-based isolation** | 6.0 | The box rule; the formal dataflow (Appendix B) | Move the Whole Box (2.14), When Unsure (2.16) |
| 0417 | Task executor preference | 6.0 | Where roomless async code runs, per task | refines (2.10) |
| 0418 | Sendable methods/keypaths | 6.0[f] | Inference from captures | (2.13) |
| 0420 | Isolation inheritance | 6.0 | Optional isolated params; `#isolation` | (2.6) |
| 0421 | AsyncSequence effects | 6.0 | Typed throws, primary assoc types · | — |
| 0423 | Dynamic isolation | 6.0 | Trap-don't-race at trust boundaries | Trap, Don't Race (2.30) |
| 0424 | checkIsolation | 6.0 | Executor-defined identity | (2.26) |
| 0430 | **sending** | 6.0 | The signed hand-off; the premise-breaker of Appendix C | Hand It Over and Let Go (2.15) |
| 043 | @isolated(any) | 6.0 | Isolation carried in the | Labels Never |

| | | | | |
|---|---|---|---|---|
| 1 | | | value—the labeled box | Lie (2.17)—its only friend |
| 0433 | Mutex | 6.0 | Checked lock-protected state | (2.2) |
| 0434 | GAI usability | 6.0[f] | Relaxations for global-actor types | (2.33) |
| 0442 | TaskGroup inference | 6.1 | · | — |
| 0449 | nonisolated cutoff | 6.1 | Types/extensions opt out of inference | (2.33) |
| 0371 | Isolated deinit | 6.2 | Deinit on the actor's executor | No Homeless Code (2.11) |
| 0457 | Attosecond Duration | 6.2 | · | — |
| 0461 | **Caller-actor default for nonisolated async** | 6.2[f] | Reverses SE-0338; `nonisolated(nonsending)` + `@concurrent` | (2.32), (2.33) |
| 0462 | Priority escalation APIs | 6.2 | Escalation events exposed | Urgency Flows Through (2.20) |
| 0463 | @Sendable handler import | 6.2 | Pessimistic import | Assume Strangers Call From Anywhere (2.28) |
| 0466 | **Default isolation control** | 6.2 | Per-module MainActor default | Simple Programs Stay Simple (2.32), Dialects (2.36) |
| 0468 | Hashable continuations | 6.2 | · | — |
| 0469 | Task naming | 6.2 | Diagnostics only · | — |

| | | | | |
|---|---|---|---|---|
| 0470 | Isolated conformances | 6.2[f] | Conformances carry rooms; SendableMetatype fence | Even Conformances Live Somewhere (2.12) |
| 0471 | isIsolatingCurrentContext | 6.2 | Refined executor identity | (2.26) |
| 0472 | Immediate tasks | 6.2 | Sync start up to first suspension | — |
| 0475 | Transactional observation | 6.2 | Emission at suspension points; anti-tearing | relies on (2.9) |
| 0481 | weak let | 6.3[f] | Weak refs may be immutable ⇒ Sendable-compatible | refines (2.13) |
| 0493 | async defer | 6.4 | Awaits in defer; inherits isolation | No Homeless Code (2.11) |
| 0504 | Cancellation shields | 6.4 | Maskable observation; flag stays monotonic | amends (2.19) |
| 0518 | ~Sendable | 6.4 | Suppress Sendable inference | (2.33) |
| 0520 | Typed-throws Task inits | 6.4 | Unused-throwing-handle warning | hygiene on (2.18) |
| 0530 | Async Result.init(catching:) | 6.4 | SE-0461 conventions in stdlib · | — |

Notable non-accepted items as of July 2026: SE-0406 AsyncStream backpressure (returned for revision; successor work in swift-async-algorithms), closure isolation control (`@inheritsIsolation`—stalled, unclear post-SE-0461), and custom main/global executors (three pitches, no acceptance).

---

### Appendix E: Glossary

- **Actor**—a room: nominal type owning an isolation domain plus a serial executor; implicitly Sendable.
- **Box**—this guide's name for an isolation region: the set of non-Sendable values connected by aliasing or reachability, which moves as a unit (2.14).

- **Carve-out**—a deliberately unsound-in-general rule justified by a specific premise, traded for ergonomics; the subject of the Ledger Rule (6.1).
- **Crossing**—a value moving between isolation domains: passed, returned, or captured across a boundary.
- **Disconnected**—a box reachable from no room; transferable (2.14).
- **Executor / serial executor**—the receptionist: a service that runs jobs; *serial* adds mutual exclusion and total order (2.8).
- **Happens-before**—the ordering relation of (2.2); a data race is a conflicting access pair without it.
- **Isolation domain**—a room: the protection boundary for mutable state—an actor instance, a global actor, a task's own values, or none (`nonisolated`).
- **Job (partial task)**—a maximal run of a task between suspension points; the unit an executor schedules.
- **Label-dropping conversion**—the Sema carve-out (from PR #62153) permitting global-actor-erasing function conversions in matching contexts; the root of the Appendix C hole family.
- **RBI**—region-based isolation: the flow-sensitive SIL pass enforcing the box rules (2.14–2.16).
- **Sema**—the AST-level type checker; owns the flow-insensitive judgments.
- **Sendable**—the memo stamp: type-level license to cross rooms freely (2.13).
- **`sending`**—the signed hand-off: value-level, flow-checked license to cross once, conferring re-send rights (2.15).
- **Squelch**—RBI's suppression of an already-computed diagnostic when isolations "match"; a carve-out that outlived its premise (Appendix C, H4).
- **Suspension point**—an `await`: where a task may yield its executor, and the granularity of interleaving (2.9).
- **Task**—a courier: one logical thread of async control flow, structured into a family tree.

---

## Appendix F: Sources

**Primary (Swift project):** the swift-evolution proposals linked per row in Appendix D; Swift Concurrency Roadmap; Approachable data-race safety vision; TSPL Concurrency; Swift 6 migration guide; Swift 6.2 release; Swift 6.3 release; Xcode build settings (Approachable Concurrency).

**Incident record:** thread 88002; thread 87519; issues #79836, #82827, #86896, #89736, #90271, #76929, #74820, #88993, #75238; PRs #62153, #86223, #90075; forum 75332.

**Secondary:** Quinn (Apple DTS), Swift Concurrency Proposal Index and Concurrency Resources; WWDC21 Swift concurrency: Behind the scenes; Milano, Turcotti & Myers, *A Flexible Type System for Fearless Concurrency*, PLDI 2022; Massicotte, concurrency glossary; mjtsai on Xcode 26.5; Lattner, Concurrency Manifesto.

**Provenance notes.** Compiler-source excerpts (`PartitionUtils.h`) are quoted via thread 87519 at commit `5b92e5c`—re-verify against `main` before citing in an evolution pitch. Attribution of the forum-75332 acknowledgment is contextually inferred. Threads 87519 and 88002 had no replies from the Swift team as of 2026-07-05. Proposal statuses for 2025–26 proposals were verified against canonical headers; SE-0327/0374 version fields rely on secondary sources.

*Prepared 2026-07-08. Corrections welcome—this document is itself an argument that such documents should exist, be maintained, and be pleasant to read.*